2026

Final Report

ARTOFEST FESTIVAL DISCOVERY AND PLANNER SERVICE

COMP2003 COMPUTING GROUP PROJECT

PROJECT NAME: ARTOFEST

CLIENT: NADEZHDA KRASTEVA / RUSSELL HOWE

GROUP: 31

NAME	ROLE
PAUL OKO-JAJA	PROJECT MANAGER
FAVOUR AKUCHIE	WEB DESIGNER
JACOB ASKEW	FRONTEND DEVELOPER
IDOWU ADELEKE	BACKEND DEVELOPER

Client Overview - Written by Paul Oko-Jaja

The artofest.com project aims to develop a comprehensive online platform that enables users to explore, plan and manage their festival experiences in a more efficient and user-friendly manner. With the increasing reliance on digital platforms for event discovery users now expect seamless, personalised and mobile-friendly experiences when searching for festivals and cultural events.

The artofest.com project was developed in collaboration with two individual clients, one of whom has a strong interest in festivals and cultural events. This helped highlight the need for a centralised platform to support event discovery and planning. Communication was primarily conducted through one client representative, whose feedback significantly influenced the system's development.

The proposed solution is a cross-platform application, accessible via both web and mobile devices, that provides users with detailed information about festivals such as location, genre, dates and related media. The platform is designed not only for dedicated festival attendees but also for more general users like individuals planning activities while travelling or on holiday.

In addition to standard browsing functionality, the platform introduces a range of interactive features that enhance user engagement. Users are able to save events of interest and share festival information with others.

Existing platforms such as FestivalFinder.eu demonstrate the demand for festival discovery services by offering extensive databases of events across Europe. However, these platforms primarily function as informational directories and lack features that support personalised user experiences, such as social sharing, streaming and integrated planning tools. This limitation creates an opportunity for artofest.com to provide a more interactive and user-focused solution.

Therefore, the primary objective of this project is to design and develop a system that not only aggregates festival information but also enhances user engagement through intuitive design and personalised functionality. By combining discovery, planning and streaming into a single platform, the system reduces the need for multiple tools and significantly improves overall usability.

Problem Statement – Written by Paul Oko-Jaja

The development of artofest.com began with a structured project planning phase, during which the team established the overall scope, objectives and approach to development. An Agile-inspired methodology was used to support an iterative and flexible development process. This approach allowed the team to break the project into manageable tasks, organised across multiple sprints and to continuously refine the system based on progress and feedback. The use of iterative planning ensured that development remained aligned with project goals while allowing for adjustments where necessary.

During the initial planning stages, key project components were defined, including system requirements, user needs and core functionality. Research into existing platforms was conducted to identify strengths and limitations within the current market, which informed the design and feature set of artofest.com. This helped establish a clear direction for the project and ensured that the proposed solution addressed real user needs.

In addition to defining the system itself, the team developed essential project management documentation. This included the creation of a work breakdown structure, risk assessment, communication plan and project timeline. These elements provided a clear framework for managing tasks, allocating responsibilities and monitoring progress throughout development.

Overall, the project planning phase laid a strong foundation for the successful development of artofest.com. By combining structured documentation with an iterative Agile approach, the team was able to maintain organisation, adapt to challenges and ensure steady progress across all stages of the project lifecycle.

Project Management – Written by Paul Oko-Jaja

Sprint Planning

The development of artofest.com was structured across eight sprints.

Sprint 1 and Sprint 2 focused on initial planning and research. During these stages, the project scope was defined, research into similar platforms was conducted and client requirements were clarified. The team also began drafting key documentation, including system requirements and early design considerations.

Sprint 3 and Sprint 4 were centred on system design and project planning. This included the creation of design documents such as system architecture, database

design and user interface layouts. Additionally, project management components such as risk assessment, communication planning and work breakdown structures were developed.

Sprint 5 focused on prototyping and early development. Initial versions of the application were created, including the home page and core user interface components. This phase allowed the team to begin translating design concepts into a functional system.

Sprint 6 and Sprint 7 represented the main development phase. Key features were implemented, including the explore page, map views, filtering functionality, API integration and database setup.

Sprint 8 was dedicated to finalisation and refinement. This included implementing authentication features, improving the user interface, completing core features and conducting final testing. Documentation and submission preparation were also completed during this sprint.

Tools and Workflow Management

To support effective collaboration and organisation throughout the development of artofest.com, a range of project management and development tools were utilised.

Trello was used as the primary project management tool, providing a visual representation of tasks organised into sprint-based columns. Each sprint contained specific tasks that were assigned to team members, allowing responsibilities to be clearly defined. This structure helped the team maintain organisation and ensured that deadlines were consistently met.

Version control was managed using GitHub, which allowed multiple team members to work on the project simultaneously without conflictsThis was particularly important during later sprints where multiple features were being developed in parallel.

In addition to task management and version control, regular communication played a key role in maintaining workflow efficiency. Team members communicated through scheduled meetings and informal discussions, ensuring that any issues were identified and resolved quickly. Progress updates were shared regularly, allowing the team to stay aligned with project goals and sprint objectives.

Risk Assessment and Mitigation

Throughout the development of artofest.com, a range of technical, organisational, and security-related risks were identified and assessed. These risks were considered during

the planning phase to minimise potential disruption to the project and ensure a stable development process.

From a technical perspective, risks such as content management system integration issues and API instability were identified. These could potentially delay feature implementation or limit system functionality, particularly when integrating external services such as travel or accommodation APIs. To mitigate these risks, a modular development approach was adopted, allowing integrations to be implemented and tested incrementally. Fallback functionality was also considered to ensure the system remained operational in the event of API failures.

Performance-related risks were also considered, particularly regarding the handling of large media content such as images and video streaming. Poor performance could negatively impact user experience through slow loading times. To address this, optimisation techniques such as efficient media handling, performance testing, and the potential use of content delivery networks were considered.

In terms of project management, risks such as missed deadlines and scope creep were identified. These could lead to delays and increased workload if not properly managed. Mitigation strategies included the use of sprint-based planning, regular progress reviews and task prioritisation. A structured approach to change management was also considered to ensure that any modifications to project scope were controlled and approved.

Human resource risks, including team member unavailability and miscommunication, were also recognised. These risks could result in slowed development or incorrect implementation of features. To minimise these issues, regular team meetings were conducted and clear communication practices were maintained. Shared documentation and cross-collaboration between team members also helped ensure continuity in development.

Finally, security risks such as data breaches or insecure API usage were considered. These risks could lead to a loss of user trust and compromise system integrity. To mitigate this, standard security practices were followed, including input validation, secure data handling, and the use of HTTPS protocols.

Client Management

Client interaction played a significant role in guiding the development of artofest.com throughout the project lifecycle. During the initial stages, key requirements were established based on the project brief and further clarified through direct communication with the client. Multiple meetings were conducted to discuss progress, gather feedback, and refine system requirements.

In addition to scheduled meetings, ongoing communication was maintained via email. This allowed the team to address queries, confirm decisions and provide regular updates between meetings. As a result, the client remained consistently informed and involved throughout the development process.

Feedback obtained from the client was incorporated into the project across multiple sprints. This iterative approach enabled the team to make continuous improvements based on client input, ensuring that the system remained aligned with both user needs and project objectives. Regular client engagement also helped to reduce misunderstandings and support more effective decision-making.

System Requirements – Written by Idowu Adeleke and Jacob Askew

Backend Perspective - Idowu Adeleke

Overview

The Artofest system is designed as a full-stack web application that enables users to explore cultural festivals, manage personalized wish lists and access streaming content associated with each festival. From a backend perspective, the system is responsible for handling data processing, business logic, authentication, and communication with the database.

The backend is implemented as a RESTful API, ensuring scalable and structured communication between the frontend client and the database layer. The system follows a modular and layered architecture to improve maintainability and extensibility.

Functional Requirements

The backend system must support the following core functionalities:

1. Festival Management

- Retrieve all festivals from the database.
- Retrieve a specific festival by ID.
- Support filtering based on:
    - Country
    - Genre
    - Art form
    - Search keyword (festival name or city)

- Return structured festival data including:
    - Basic details (name, city, country)
    - Main image (`image_url`)
    - Additional images (`festival_images`)
    - Website link
- Aggregate multiple festival images into a JSON array using database-level processing.

2. User Authentication

- Allow users to register with:
    - Username
    - Email
    - Password
- Allow users to log in securely.
- Generate and return a JWT (JSON Web Token) upon successful login.
- Protect secured endpoints using token-based authentication.

3. Wishlist Management

- Allow authenticated users to:
    - Add festivals to their wishlist.
    - View all saved festivals.
    - Remove festivals from their wishlist.
- Ensure that wishlist data is linked to user profiles.
- Return enriched festival data within wishlist responses.

4. Streaming Content

- Retrieve streaming content grouped by:
    - Festival
    - Category (e.g., Performance, Music, Talks)
- Allow retrieval of streams for a specific festival.
- Return structured grouped JSON responses to simplify frontend rendering.

5. Image Handling

- Store festival images in a separate relational table (`festival_images`).
- Associate multiple images with a single festival (one-to-many relationship).
- Aggregate images dynamically using:

    - PostgreSQL `json_agg()` function
- Serve static image files via backend public directory.

Non-Functional Requirements

1. Performance

- API responses must be efficient and optimized.
- Database queries must use joins and aggregation effectively.
- Minimize redundant queries using grouped SQL operations.

2. Scalability

- The system must support future expansion, including:
    - Additional festivals
    - More users
    - Increased image storage
- Database normalization ensures scalability.
- REST architecture allows easy integration with additional services.

3. Security

- Use JWT-based authentication for protected routes.
- Encrypt user passwords before storage.
- Restrict access to sensitive endpoints (e.g., wishlist).
- Prevent SQL injection using parameterized queries.

4. Maintainability

- Code is structured using controllers and modular routing.
- Separation of concerns between:
    - API routes
    - Business logic
    - Database queries
- Easy to extend with new endpoints or features.

5. Reliability

- System must handle errors gracefully.
- Return appropriate HTTP status codes:
    - `200` – Success

- `400` – Bad request
- `401` – Unauthorized
- `404` – Not found
- `500` – Server error

6. Usability (API Level)

- API must follow RESTful standards.
- Responses must be consistent and predictable.
- JSON structure must be frontend-friendly (e.g., grouped streams, aggregated images).

2.4 System Constraints

- Backend is deployed on a cloud platform (Render).
- Uses PostgreSQL as the relational database.
- Static images are served from a public directory.
- API follows OpenAPI (Swagger) documentation standards.

2.5 API-Based Requirement Validation

The implemented API endpoints (as shown in Swagger documentation) confirm that all system requirements are met:

- Festival endpoints → data retrieval and filtering
- Auth endpoints → user management
- Wishlist endpoints → personalization
- Stream endpoints → media access

Additionally, the inclusion of the `festival_images` field demonstrates enhanced functionality beyond basic requirements by supporting multiple images per festival.

Summary

The backend system successfully fulfils all functional and non-functional requirements by providing a secure, scalable, and efficient API. The integration of advanced database features such as JSON aggregation, combined with RESTful design principles, ensures that the system is both robust and adaptable for future enhancements.

Frontend Perspective – Jacob Askew

From the frontend perspective, the ArtoFest system was designed and implemented as a cross-platform application using React Native with Expo and Expo Router. This approach was chosen because it allowed the same core codebase to support both web and mobile usage, which directly aligned with the project objective of creating a responsive festival discovery platform that could work across different devices. The frontend acts as the user-facing layer of the system and is responsible for presenting festival data, handling navigation, supporting account-based features and connecting users to the backend API

The frontend was required to provide a clear and accessible way for users to browse, search, view, save, and interact with festival information. The application therefore includes a homepage, an explore page, individual festival detail pages, account pages, a planning page, a streaming page, and a booking/travel ideas page. These screens are connected using Expo Router, allowing the application to behave like a multi-page web application while still being suitable for mobile app development.

A key frontend requirement was dynamic festival data retrieval. Rather than hard-coding festival information into the interface, the frontend retrieves festival records from the backend API. The shared API base URL is stored centrally and festival-related helper functions are used to request all festivals or a specific festival by ID. This improves maintainability because changes to festival data can be made through the backend/database without requiring the frontend code to be rewritten. The frontend then normalises the data before rendering it, ensuring that missing or inconsistent values do not break the interface.

The homepage was designed to act as the first user entry point into the platform. It includes a visual hero section, dropdown-based search controls and festival result cards. The search system allows users to filter festivals by country, month, and art form. These filters were implemented using local React state and derived option lists generated from the API response. This means the available dropdown options are based on the actual festival data returned by the backend, rather than fixed manually. The frontend also validates the search interaction by only showing results once at least one filter has been selected, which prevents the page from becoming overcrowded with unnecessary results.

The explore page provides a broader browsing experience. It retrieves festival data from the backend and presents it through country-based groupings, search and filtering controls, summary statistics and an interactive map. On the web version, the map uses Leaflet through a dedicated web-specific map component, while the native version uses a separate native map implementation. This separation was necessary because web and mobile platforms handle maps differently. By creating platform-specific map

components, the frontend could provide equivalent functionality while avoiding compatibility issues between web and native environments.

The individual festival detail page allows users to open a specific festival from other areas of the application and view more detailed information. This page uses the festival ID passed through routing, fetches the matching festival from the backend, and displays information such as the name, image, location, date, art form, website link, and description. This supports the requirement for each festival to have its own profile-style page rather than only appearing inside a list.

Authentication was also implemented on the frontend to support account-based functionality. Users can register and log in through separate pages. The login process sends the user's credentials to the backend authentication endpoint and expects a token-based response. Once authenticated, the session is stored locally. On the web version, localStorage is used, while on native platforms Expo SecureStore is used. This platform-aware storage design allows the application to preserve the user's login state while using a storage method suitable for each environment.

The planning page is one of the most important frontend features because it connects account authentication with personalised festival saving. Users who are not logged in are shown a message explaining that an account is required before they can use the planning tools. Once logged in, the page uses the stored bearer token to request the user's wishlist from the backend. The user can search for festivals, add a festival to their wishlist, remove saved festivals, filter saved festivals by month, open festival websites, and generate a printable wishlist summary on the web version. This design ensures that wishlist data is linked to authenticated users rather than being stored only in the browser.

The frontend also includes a streaming page that retrieves stream data from the backend and transforms grouped API responses into stream cards that can be displayed clearly to users. The frontend handles different stream platforms by normalising platform values and, where possible, generating YouTube thumbnail previews from stream URLs. This improves the visual quality of the page and makes the streaming feature more engaging without requiring every stream to have a manually stored thumbnail.

A custom top navigation bar and global footer were implemented to create a consistent layout across the application. The navigation bar provides access to the main areas of the system, including Home, Explore, Plan, Book, Stream, and Profile. It adapts between desktop and mobile layouts by using screen width detection. On desktop, the navigation is distributed horizontally, while on mobile it becomes a more compact layout with horizontally scrollable navigation buttons. The footer provides supporting links such as About, FAQ, legal pages, and key feature links. This improves the overall

structure of the application and makes it feel more like a complete product rather than a set of disconnected screens.

Several defensive programming techniques were used throughout the frontend. API responses are checked before rendering, missing images are replaced with fallback elements, website links are normalised to include a valid protocol, and invalid or missing festival IDs are filtered out before navigation. Loading states and error messages are also used so that users receive feedback when data is being fetched or when a request fails. These decisions improve reliability and prevent common frontend errors from damaging the user experience.

Overall, the frontend successfully implements the main user-facing functionality of ArtoFest. It provides a responsive interface for festival discovery, integrates with the backend API, supports account-based wishlist planning, displays festival details, presents streaming content, and maintains consistent navigation across pages. Although some planned features, such as a fully functional AI planner and real booking integration, were not completed within the available time and free resource constraints, the frontend provides a functional MVP that demonstrates the intended direction of the platform and supports further development after the project submission.

Project Overview – Written by Favour Akuchie

The ArtoFest project is a user-focused festival discovery and planning platform designed to simplify how users find and organise cultural events. The system brings together festival browsing, personalised planning, and supporting features such as streaming and travel inspiration into a single, unified platform.

From a design perspective, the project aims to provide a clean, intuitive, and visually engaging interface that supports both casual browsing and structured planning. Users can explore festivals through search filters, grid layouts, and map-based views, allowing them to discover events based on location, time, and art form.

A key objective of the platform is to reduce the fragmentation commonly experienced when planning festival activities. Instead of relying on multiple websites for discovery, scheduling, and information gathering, ArtoFest centralises these features into one cohesive system. This improves usability and enhances the overall user experience.

The platform is designed as a cross-platform application, ensuring accessibility across both web and mobile devices. This approach supports a wider audience and aligns with modern user expectations for responsive and flexible digital services.

User Stories – Written by Favour Akuchie

User stories were developed to capture the key interactions and expectations of users within the ArtoFest system. These stories helped guide both design and development decisions by focusing on real user needs.

Festival Discovery

- As a user, I want to search for festivals by country, month, or art form so that I can easily find events that match my interests.
- As a user, I want to browse festivals in a grid or map view so that I can explore events visually.

Festival Details

- As a user, I want to view detailed information about a festival so that I can understand what it offers before deciding to attend.
- As a user, I want to access a festival's official website so that I can get more information or make further plans.

User Accounts and Planning

- As a user, I want to create an account so that I can save festivals I am interested in.
- As a user, I want to add festivals to a wishlist so that I can plan my schedule.
- As a user, I want to remove festivals from my wishlist so that I can update my plans.

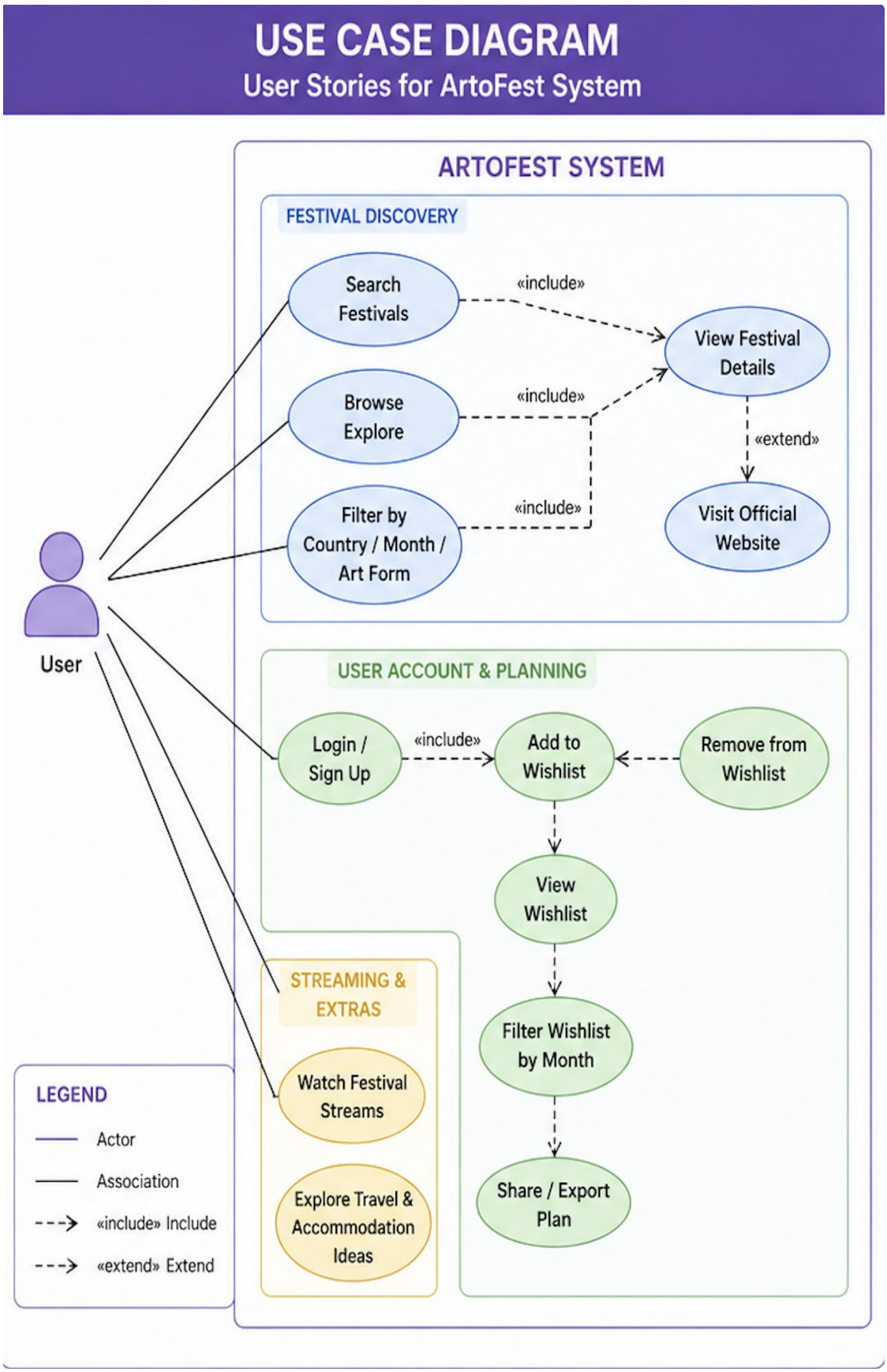
Filtering and Navigation

- As a user, I want simple navigation across pages so that I can move between features easily.
- As a user, I want filters to refine results so that I am not overwhelmed with too many options.

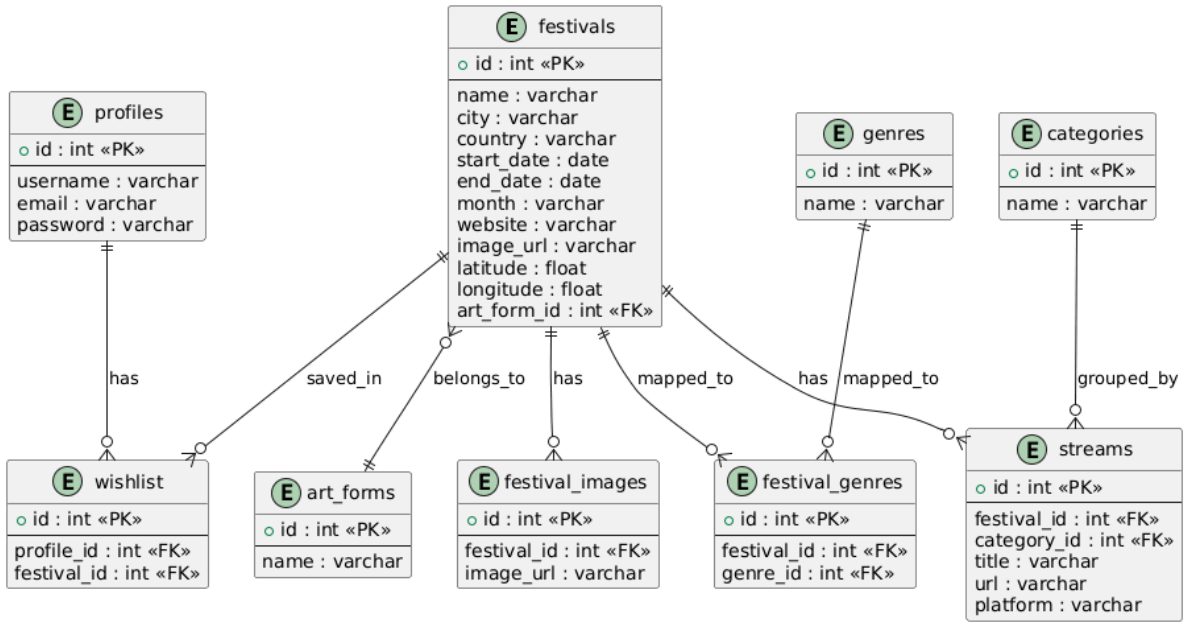
Streaming and Additional Features

- As a user, I want to watch festival-related streams so that I can experience events remotely.
- As a user, I want to explore travel and accommodation ideas so that I can plan my trip more effectively.

These user stories ensured that the system remained focused on usability, accessibility, and real-world application.



ERD (Database Design) - Written by Idowu Adeleke



Overview of Database Design

The Artofest system uses a relational database (PostgreSQL) designed to efficiently manage festival data, user interactions, and media content. The database structure follows normalization principles to reduce redundancy while maintaining scalability and performance.

The design supports complex relationships such as:

- One-to-many (festival → images)
- Many-to-many (festival ↔ genres)
- One-to-many (user → wishlist)

Core Tables Explanation

Festivals Table

This is the central table in the system.

It stores:

- Festival name, city, country
- Main image (`image_url`)
- Website link
- Coordinates (latitude & longitude)
- Art form reference (`art_form_id`)

This table acts as the **primary entity** connected to most other tables.

Festival Images Table

This table stores **multiple images per festival**, enabling richer visual representation.

- Each record contains:
  - `festival_id` (foreign key)
  - `image_url`

Relationship:

- One festival → many images

Genres & Festival_Genres Tables

A many-to-many relationship is implemented using a junction table:

- `genres` → stores genre names
- `festival_genres` → links festivals and genres

Why this design?

- A festival can belong to multiple genres
- A genre can apply to multiple festivals

Art Forms Table

Stores categories such as:

- Music
- Film
- Theatre
- Digital Arts

Each festival references one art form via `art_form_id`.

Wishlist Table

Stores user preferences.

- Links:
  - `profile_id`

    - `festival_id`

Relationship:

- One user → many wishlist items

Streams & Categories Tables

Used to organize streaming content.

- Streams are grouped by:
    - Festival
    - Category (e.g., Performance, Talks)

Design Decisions

Normalization

The database is normalized to:

- Avoid duplicate data
- Improve consistency
- Support scalability

Separation of Images

Instead of storing multiple images in one column:

- A separate `festival_images` table is used

This enables:

- Flexible image storage
- Efficient querying
- Cleaner database structure

Use of Foreign Keys

Foreign key constraints ensure:

- Data integrity

- Consistent relationships between tables

Scalability Considerations

The database design supports:

- Adding more festivals without structural changes
- Expanding image storage easily
- Introducing new categories or features

Summary

The ERD demonstrates a well-structured relational database that balances normalization, performance, and flexibility. The design ensures efficient querying and supports advanced features such as multi-image handling and categorized streaming content.

Backend System Design - Written by Idowu Adeleke

Architecture Overview

The backend system follows a **layered architecture (MVC-inspired)**:

- **Routes Layer** → Handles API endpoints
- **Controller Layer** → Contains business logic
- **Database Layer** → Executes SQL queries

This structure improves:

- Maintainability
- Scalability
- Code organization
-

Technology Stack

The backend is implemented using:

- **Node.js** → Server runtime
- **Express.js** → API framework
- **PostgreSQL** → Relational database
- **JWT** → Authentication

- **Swagger (OpenAPI)** → API documentation

Why these choices?

- Node.js → asynchronous, fast, scalable
- PostgreSQL → strong relational capabilities
- Express → lightweight and flexible
- JWT → secure stateless authentication

API Design

The system exposes RESTful endpoints such as:

- `/api/festivals`
- `/api/auth/login`
- `/api/wishlist`
- `/api/streams`

Each endpoint:

- Follows REST conventions
- Returns structured JSON responses
- Uses appropriate HTTP methods (GET, POST, PUT, DELETE)

Database Integration

The backend communicates with PostgreSQL using parameterized queries to ensure security and prevent SQL injection.

```
// controllers/festivalController.js
const pool = require('../db/connection');

const getFestivals = async (req, res) => {
  try {
    const { country, genre, art_form, search } = req.query;

    let query = `
      SELECT
        f.*,
        af.name AS art_form,
        COALESCE(
          json_agg(DISTINCT fi.image_url) FILTER (WHERE fi.image_url IS NOT NULL),
          '[]'
        ) AS festival_images
      FROM festivals f
      LEFT JOIN art_forms af ON f.art_form_id = af.id
      LEFT JOIN festival_genres fg ON f.id = fg.festival_id
      LEFT JOIN genres g ON fg.genre_id = g.id
      LEFT JOIN festival_images fi ON f.id = fi.festival_id
      WHERE 1=1
    `;
```

This query above demonstrates:

- Use of joins to combine multiple tables
- Aggregation of images into a JSON array
- Efficient data retrieval in a single query

JSON Aggregation (Key Feature)

A key feature of the backend is the use of PostgreSQL's `json_agg()` function.

This allows:

- Multiple image records to be returned as:

`"images": ["url1", "url2", "url3"]`

Instead of:

- Multiple rows per festival

```
COALESCE(
  json_agg(fi.image_url) FILTER (WHERE fi.image_url IS NOT NULL),
  '[]'
) AS festival_images
```

- Prevents null values
- Ensures consistent API responses
- Improves frontend usability
-

Authentication System

The backend uses JWT-based authentication.

Process:

1. User logs in
2. Server validates credentials
3. Token is generated
4. Token is used for protected routes

middleware > JS authMiddleware.js > ...

```
1   const jwt = require('jsonwebtoken');
2
3   const authMiddleware = (req, res, next) => {
4     const authHeader = req.headers.authorization;
5
6     if (!authHeader) {
7       return res.status(401).json({ message: "No token provided" });
8     }
9
10    const token = authHeader.split(" ")[1];
11
12    try {
13      const decoded = jwt.verify(token, process.env.JWT_SECRET);
14      req.user = decoded;
15      next();
16    } catch (error) {
17      return res.status(401).json({ message: "Invalid token" });
18    }
19  };
20
21  module.exports = authMiddleware;
```

Fig 3.0 (Screenshot of code snippet from AuthMiddleware used to generate JWT token)

Error Handling

The backend includes structured error handling:

- Try-catch blocks in controllers
- Meaningful HTTP responses
- Logging for debugging

As shown in Fig3.0,

```
catch (error) {
  console.error(error);
  res.status(500).json({ message: "Server Error" });
}
```

System Efficiency

The backend is optimized by:

- Using joins instead of multiple queries
- Aggregating data at the database level
- Reducing API response size

Overview of API Architecture

The Artofest backend exposes a RESTful API designed to provide structured access to festival data, user authentication, wishlist management, and streaming content.

The API follows a **layered architecture**, consisting of:

- **Routing Layer** – Handles HTTP requests and maps endpoints to controllers
- **Controller Layer** – Contains business logic and processes requests
- **Database Layer** – Executes SQL queries using PostgreSQL

All responses are returned in **JSON format**, ensuring compatibility with the frontend application.

The API was documented using **Swagger (OpenAPI 3.0)**, which provides interactive testing and clear endpoint definitions.

Festival Endpoints

GET /api/festivals

Purpose

Retrieves all festivals with optional filtering capabilities such as country, genre, art form, and search keywords.

Request Parameters

Parameter	Type	Description

country	string	Filter festivals by country
genre	string	Filter by genre
art_form	string	Filter by art form
search	string	Search by name or city

Response Structure

- Returns an array of festival objects
- Includes aggregated images using `json_agg`

Example Response

```
[
  {
    "id": 1,
    "name": "Eurocks",
    "city": "Cravanche",
    "country": "France",
    "image_url": "...",
    "images": [
      "image1.jpg",
      "image2.jpg"
    ]
  }
]
```

Implementation Explanation

This endpoint uses multiple **LEFT JOINs** to combine data from:

- `festivals`
- `art_forms`
- `genres`
- `festival_images`

The use of **json_agg()** allows multiple related images to be grouped into a single JSON array.

```
// controllers/festivalController.js
const pool = require('../db/connection');

const getFestivals = async (req, res) => {
  try {
    const { country, genre, art_form, search } = req.query;

    let query = `
      SELECT
        f.*,
        af.name AS art_form,
        COALESCE(
          json_agg(DISTINCT fi.image_url) FILTER (WHERE fi.image_url IS NOT NULL),
          '[]'
        ) AS festival_images
      FROM festivals f
      LEFT JOIN art_forms af ON f.art_form_id = af.id
      LEFT JOIN festival_genres fg ON f.id = fg.festival_id
      LEFT JOIN genres g ON fg.genre_id = g.id
      LEFT JOIN festival_images fi ON f.id = fi.festival_id
      WHERE 1=1
    `;

    const values = [];
    let index = 1;

    if (country) {
      query += ` AND f.country ILIKE $${index}`;
      values.push(`%${country}%`);
      index++;
    }

    if (genre) {
      query += ` AND g.name ILIKE $${index}`;
      values.push(`%${genre}%`);
      index++;
    }

    if (art_form) {
      query += ` AND af.name ILIKE $${index}`;
      values.push(`%${art_form}%`);
      index++;
    }

    if (search) {
      query += ` AND (
```

Design Justification

- Supports flexible filtering without multiple endpoints
- Reduces frontend complexity
- Efficient data retrieval using a single query

GET /api/festivals/{id}

Purpose

Retrieves detailed information for a specific festival.

Request

- Path parameter: `id`

Response

Returns a single festival object including:

- Full festival details
- Aggregated images array

Implementation Explanation

- Uses `WHERE f.id = $1`
- Aggregates related images using `json_agg`
- Groups results to avoid duplication

```
const getFestivalById = async (req, res) => {
  const { id } = req.params;

  try {
    const result = await pool.query(
      `
      SELECT
        f.*,
        af.name AS art_form,
        COALESCE(
          json_agg(DISTINCT fi.image_url) FILTER (WHERE fi.image_url IS NOT NULL),
          '[]'
        ) AS festival_images
      FROM festivals f
      LEFT JOIN art_forms af ON f.art_form_id = af.id
      LEFT JOIN festival_images fi ON f.id = fi.festival_id
      WHERE f.id = $1
      GROUP BY f.id, af.name
      `,
      [id]
    );

    if (result.rows.length === 0) {
      return res.status(404).json({ message: "Festival not found" });
    }

    res.json(result.rows[0]);

  } catch (error) {
    console.error(error);
    res.status(500).json({ message: "Server Error" });
  }
};
```

Error Handling

- Returns **404** if festival is not found
- Returns **500** for server/database errors

Authentication Endpoints

POST /api/auth/register

Purpose

Registers a new user account.

Request Body

```
{
  "username": "user1",
  "email": "user@email.com",
  "password": "securepassword"
}
```

Implementation Explanation

- Validates input fields
- Stores user credentials in the database
- Passwords should be hashed (e.g., bcrypt)

```
exports.register = async (req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  const { username, email, password } = req.body;

  try {
    const hashedPassword = await bcrypt.hash(password, 10);

    const result = await pool.query(
      "INSERT INTO profiles (username, email, password_hash) VALUES ($1,$2,$3) RETURNING id, username, e
      [username, email, hashedPassword]
    );

    res.status(201).json({
      message: "User registered successfully",
      user: result.rows[0],
    });

  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

POST /api/auth/login

Purpose

Authenticates users and returns a JWT token.

Response

```
{
  "token": "jwt_token_here"
}
```

Implementation Explanation

- Verifies email and password
- Generates JWT token
- Token used for protected routes

```
exports.login = async (req, res) => {
  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return res.status(400).json({ errors: errors.array() });
  }

  const { email, password } = req.body;

  try {
    const result = await pool.query(
      "SELECT * FROM profiles WHERE email = $1",
      [email]
    );

    if (result.rows.length === 0) {
      return res.status(401).json({ message: "Invalid credentials" });
    }

    const user = result.rows[0];

    const validPassword = await bcrypt.compare(password, user.password_hash);

    if (!validPassword) {
      return res.status(401).json({ message: "Invalid credentials" });
    }

    const token = jwt.sign(
      { id: user.id },
      JWT_SECRET,
      { expiresIn: "1d" }
    );

    res.json({
      message: "Login successful",
      token,
      user: {
        id: user.id,
        username: user.username,
        email: user.email
      }
    });

  } catch (err) {
    res.status(500).json({ error: err.message });
```

Design Justification

- JWT enables **stateless authentication**
- Improves scalability
- Secures protected endpoints

Wishlist Endpoints

GET /api/wishlist

Purpose

Retrieves all festivals saved by a user.

Security

- Requires JWT authentication

Response

Returns a list of wishlist items including:

- Festival name
- Country
- Image
- Website

controllers > JS wishlistController.js > ...

```
1   const pool = require("../db/connection");
2
3   exports.getWishlist = async (req, res) => {
4     const userId = req.user.id;
5
6     try {
7       const result = await pool.query(
8         `SELECT f.*
9          FROM wishlist w
10         JOIN festivals f ON f.id = w.festival_id
11         WHERE w.profile_id = $1`,
12        [userId]
13      );
14
15      res.json(result.rows);
16
17    } catch (err) {
18      res.status(500).json({ error: err.message });
19    }
20  };
21
```

POST /api/wishlist

Purpose

Adds a festival to the user's wishlist.

Request

```
{
  "festival_id": 5
}
```

DELETE /api/wishlist/{festivalId}

Purpose

Removes a festival from the wishlist.

Implementation Explanation

- Uses user ID from JWT
- Prevents duplicate entries
- Ensures data integrity

Stream Endpoints

GET /api/streams

Purpose

Retrieves all streams grouped by:

- Festival
- Category

Response Structure

```
[
  {
    "festival_id": 4,
```

```
    "festival_name": "Avignon Festival",
    "categories": [
      {
        "name": "Performance",
        "streams": [
          {
            "title": "Live Show",
            "url": "...",
            "platform": "youtube"
          }
        ]
      }
    ]
  }
]
```

```
const pool = require('../db/connection');

// GET ALL STREAMS (GROUPED)
exports.getAllStreams = async (req, res) => {
  try {
    const result = await pool.query(`
      SELECT
        s.id,
        s.title,
        s.url,
        s.platform,
        f.id AS festival_id,
        f.name AS festival_name,
        c.name AS category
      FROM streams s
      JOIN festivals f ON s.festival_id = f.id
      JOIN stream_categories c ON s.category_id = c.id
      ORDER BY f.name, c.name;
    `);

    const grouped = {};

    result.rows.forEach(row => {
      const festId = row.festival_id;

      if (!grouped[festId]) {
        grouped[festId] = {
          festival_id: festId,
          festival_name: row.festival_name,
          categories: {}
        };
      }

      if (!grouped[festId].categories[row.category]) {
        grouped[festId].categories[row.category] = [];
      }

      grouped[festId].categories[row.category].push({
        id: row.id,
        title: row.title,
        url: row.url,
        platform: row.platform
      });
    });
```

GET /api/streams/{festivalId}

Purpose

Retrieves streams for a specific festival.

Implementation Explanation

- Groups streams by category
- Ensures structured frontend rendering

Error Handling and Validation

The API implements consistent error handling:

Status Code	Meaning
200	Success
201	Resource created
400	Bad request
401	Unauthorized
404	Not found
500	Server error

Validation includes:

- Required fields checking
- Parameter validation
- Secure authentication

API Design Decisions

Key design decisions include:

Use of REST Principles

- Clear resource-based endpoints
- Standard HTTP methods (GET, POST, DELETE)

Use of JSON Aggregation

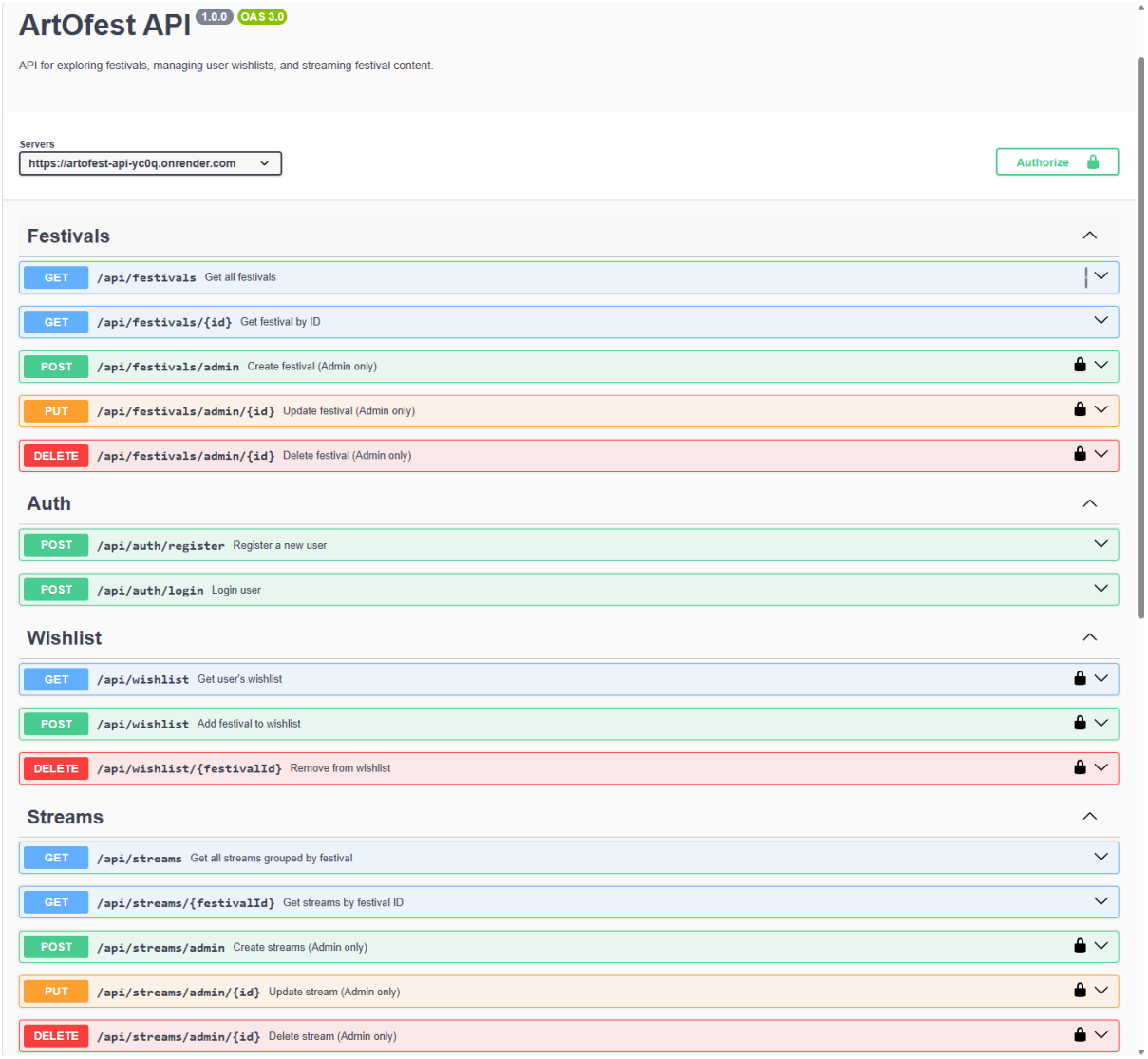
- Reduces multiple API calls
- Improves frontend performance

JWT Authentication

- Stateless and scalable
- Secure access control

Swagger Documentation

- Provides interactive API testing
- Improves developer experience



The Screenshot above shows the final API structure and providing Content

management system to allow insertion of festival by an admin without having to go through the whole backend system.

Content Management System (CMS) – Admin Functionality

Written by: Idowu Adeleke (Backend Developer)

Overview

The system includes a Content Management System (CMS) that allows administrators to manage festival and stream data dynamically through secure backend endpoints. Unlike public endpoints, CMS functionality is restricted to authenticated users with administrative privileges. This ensures that only authorized users can create, update, or delete critical system data. In the current frontend implementation, the Admin Section button is visible from the Profile page; however, this button alone does not grant administrative access. Create, update and delete requests are still checked by the backend, and only accounts that have been granted admin permission by the server can successfully modify festival or stream data.

Authentication & Access Control

CMS endpoints are protected using:

- JWT Authentication
- Role-Based Authorization (Admin Only)

Implementation

1. User logs in and receives a JWT token
2. Token is included in request headers:

Authorization: Bearer <token>

3. Middleware verifies:
    a. Token validity (authenticate)
    b. Admin role (isAdmin)

Design Justification

- Prevents unauthorized data manipulation
- Ensures secure system operations
- Demonstrates implementation of access control mechanisms

Festival Management (CMS)

POST /api/festivals/admin

Purpose

Allows an administrator to create a new festival including all related data such as:

- Art form
- Genres
- Images
- Metadata (location, description, dates)

Request Structure

```
{
 "name": "Berlin Soundwave Festival",
 "city": "Berlin",
 "country": "Germany",
 "website": "https://example.com",
 "image_url": "https://...",

 "latitude": 52.52,
 "longitude": 13.405,

 "description": "Festival description",
 "start_date": "2026-08-12",
 "end_date": "2026-08-16",
 "month": "August",

 "art_form": "Music",

 "genres": ["Electronic", "Techno"],

 "images": [
  "image1.jpg",
  "image2.jpg"
 ]
}
```

Implementation Explanation

This endpoint performs a multi-step transactional operation:

1. Checks if art_form exists → inserts if not
2. Inserts festival into festivals table
3. Inserts or retrieves genres
4. Links festival to genres (festival_genres)
5. Inserts multiple images (festival_images)

All operations are wrapped in:
BEGIN → COMMIT / ROLLBACK

Design Justification

- Ensures atomicity (all operations succeed or fail together)
- Maintains data integrity across multiple tables
- Demonstrates advanced use of relational database transactions

PUT /api/festivals/admin/{id}

Purpose
Updates an existing festival and its related data.

Implementation Explanation

- Updates core festival fields
- Deletes existing relationships:
    - Genres
    - Images
- Reinserts updated data

Design Justification

- Ensures data consistency
- Simplifies update logic
- Avoids complex partial updates

DELETE /api/festivals/admin/{id}
Purpose
Deletes a festival from the system.

Implementation Explanation

- Removes festival record
- Related records are handled via:
    - Cascading deletes OR manual cleanup

Design Justification

- Maintains database cleanliness
- Prevents orphaned records

Stream Management (CMS)

POST /api/streams/admin

Purpose

Allows admin to create multiple streams linked to a festival.

Request Structure

```
{
 "festival_id": 1,
 "streams": [
  {
   "title": "Live Opening Ceremony",
   "url": "https://youtube.com/...",
   "platform": "youtube",
   "category_id": 1
  }
 ]
}
```

Implementation Explanation

- Iterates through stream array
- Inserts each stream into streams table
- Links each stream to:
    - Festival
    - Category

Design Justification

- Supports bulk insertion
- Improves efficiency for admin users
- Reduces number of API calls

PUT /api/streams/admin/{id}

Updates a specific stream.

DELETE /api/streams/admin/{id}

Deletes a stream from the system.

Validation and Security

The CMS endpoints implement:

- Input validation (required fields, data types)
- Parameterized queries (prevents SQL injection)
- JWT authentication
- Role-based access control

Error Handling

Status Code	Meaning
400	Invalid input
401	Unauthorized
403	Forbidden (not admin)
404	Resource not found
500	Server error

System Design Considerations

The CMS was designed with the following principles:

1. Separation of Concerns
    - Public API → read-only access
    - CMS API → data management

2. Data Integrity
    - Transactions ensure consistency
    - Relationships are strictly maintained

3. Scalability
    - Modular design allows future expansion (e.g., admin dashboard UI)

Frontend Architecture – Written by Jacob Askew

Overview

The frontend of ArtoFest was implemented using React Native with Expo and Expo Router. This architecture was selected because the project required a system that could support both web and mobile usage from a shared codebase. Instead of building a separate website and separate native mobile applications, React Native and Expo allowed the frontend to be structured as a single application that can run on the web while also remaining suitable for iOS and Android development.

The frontend is responsible for the complete user-facing experience of the system. This includes the homepage, festival search, explore page, map view, individual festival profile pages, user authentication screens, profile page, wishlist planning page, streaming page, booking/travel ideas page, and supporting information pages such as About, FAQ, Privacy Policy and Terms of Service.

The frontend communicates with the backend using REST API requests. Festival data, authentication, wishlist information and stream content are not hard-coded into the application. Instead, the frontend retrieves this data dynamically from the backend API and renders it through reusable components and page-level state management. This makes the system easier to maintain because the backend/database can be updated without requiring major changes to the frontend interface.

Technology Stack

The frontend uses the following main technologies:

React Native was used to build the user interface using reusable components. This allowed the application to be structured into screens, views, cards, buttons, forms and layout sections that could be reused across different parts of the system.

Expo was used as the development framework because it simplifies React Native development and supports running the project on web and mobile platforms. Expo also provides access to useful platform features such as secure storage, routing support, and simplified testing through Expo development tools.

Expo Router was used for file-based navigation. Each page in the application is represented by a file inside the app directory. This made the routing structure easier to understand because the page structure of the application is reflected directly in the folder structure.

TypeScript was used to improve reliability and maintainability. By defining expected data structures such as festival records, authentication sessions and wishlist items, the frontend reduces the risk of using incorrect values from the API.

React hooks such as useState, useEffect, useMemo and context hooks were used to manage page state, API loading, filtering, authentication state and derived data.

Leaflet and React-Leaflet were used for the web-based map implementation on the Explore page. A separate native map component was also included for mobile compatibility using react-native-maps.

Expo SecureStore and localStorage were used for authentication session storage. On native platforms, Expo SecureStore provides more suitable secure storage. On the web version, localStorage is used because SecureStore is not available in the same way in browsers.

Application Structure

The frontend follows the structure of an Expo Router application. The main application layout is defined in app/_layout.tsx. This layout wraps the application in the authentication provider, displays the custom top navigation bar, and renders the currently active page using Expo Router's Slot component.

This structure is important because it means that global elements do not need to be manually repeated on every page. The top navigation and authentication context are available throughout the application, which improves consistency and reduces duplicated code.

The main user-facing pages are stored inside the app/(tabs) directory. Although the project originally came from an Expo tab template, the default tab navigation was effectively replaced by a custom top navigation system. The (tabs)/_layout.tsx file now mainly provides a safe-area wrapper and renders child pages through Slot. This means that navigation is controlled by the custom top navigation bar rather than the default Expo tab bar.

The main pages include:

- index.tsx for the homepage and festival search
- explore.tsx for country browsing, filters and map-based exploration
- plan.tsx for wishlist planning, month selection and sharing
- book.tsx for travel and accommodation idea mock-ups
- stream.tsx for festival stream content
- profile.tsx for account status and logout
- login.tsx and signup.tsx for authentication
- festival/[id].tsx for individual festival detail pages

Supporting pages are also included for About, FAQ, legal information and booking-unavailable messaging.

Global Layout and Navigation

A key design decision was to implement a custom top navigation bar instead of relying on the default Expo tab navigation. This was done because the project needed to behave more like a professional website on desktop while still working on mobile screens.

The custom navigation is implemented in components/top-nav.tsx. It provides links to the main areas of the application, including Home, Explore, Plan, Book, Stream and Profile. The navigation adapts depending on screen size. On desktop, the logo is positioned on the left, the page links are distributed across the centre, and the profile

icon is positioned on the right. On mobile, the layout becomes more compact and the navigation items are displayed in a horizontally scrollable row.

This responsive behaviour was important because the project needed to meet the requirement for a user-friendly cross-platform interface. A navigation layout that works well on desktop can become too wide or difficult to use on mobile, so the layout had to change depending on the available screen width.

A global footer was also implemented in components/footer.tsx. The footer includes the ArtoFest branding, short platform description, internal feature links, About links, FAQ links, legal links and contact information. This footer is used across the main pages to improve consistency and make the application feel like a complete product rather than a collection of individual screens.

API Integration

The frontend communicates with the backend through a central API base URL stored in lib/api.ts. This avoids repeating the full backend URL across multiple files. If the backend URL changes in the future, it can be updated in one location rather than in every page.

Festival-related API logic is supported by lib/festivals.ts. This file defines the expected festival record structure and contains helper functions for retrieving all festivals and retrieving a festival by ID. It also includes utility functions for formatting dates, normalising month values, building festival locations, validating website URLs and converting IDs into numeric values.

This separation improves maintainability because festival data handling is not spread randomly across every screen. Pages such as the homepage and festival detail page can use shared helper functions instead of duplicating the same logic.

The frontend uses API integration for several major features:

- The homepage retrieves festivals from the backend and allows users to filter them by country, month and art form.
- The Explore page retrieves festivals and displays them through country groupings, filter controls and a map.
- The festival detail page retrieves one festival by ID and displays a detailed profile.
- The login and signup pages communicate with the backend authentication endpoints.
- The Plan page communicates with the wishlist endpoints using a bearer token.
- The Stream page retrieves grouped stream content from the backend and renders it as stream cards.

This API-based structure means that the frontend is not simply a static mock-up. It is connected to live backend functionality and responds to data returned from the database.

Data Handling and Normalisation

Because API data can sometimes be incomplete, inconsistent or returned in slightly different formats, the frontend includes data normalisation logic before rendering information to users.

For example, festival names, countries, cities, art forms and website values are normalised before display. Empty values are handled safely so that missing data does not break the page layout. Website links are passed through a helper function that adds https:// if the backend value does not already include a protocol. This prevents broken links when users open official festival websites.

Month handling was also normalised. The backend may provide a month_text value or a start date. The frontend therefore includes logic to convert month abbreviations and date values into consistent month names. This is used in the homepage search filters and the planning page month selector.

Festival IDs are also checked before navigation. When a user opens a festival detail page, the frontend converts the ID into a numeric value and prevents invalid IDs from being used. This reduces the risk of routing errors.

This defensive approach makes the frontend more reliable because it does not assume that every API response will always be perfect.

Homepage Architecture

The homepage acts as the main entry point into the application. It is implemented in app/(tabs)/index.tsx.

The page loads festival data from the backend using the shared fetchFestivals() function. Once the data is loaded, the homepage uses local state to store the selected country, selected month, selected art form and dropdown state. It also uses a hasSearched state value so that search results are only displayed after the user actively performs a search.

The dropdown options are generated from the festival data rather than being hard-coded. This means that if the backend adds a new country or art form, the frontend can include it automatically in the filter options. This improves scalability and reduces manual maintenance.

The homepage also includes festival cards. Each festival card displays key information such as the image, name, location, date and art form. Users can open individual festival

detail pages by selecting a festival card. This creates a clear user journey from discovery to detailed information.

Explore Page Architecture

The Explore page is implemented in app/(tabs)/explore.tsx and provides a more detailed browsing experience than the homepage.

This page retrieves festival data from the backend and maps the API response into a frontend-friendly structure. It includes logic for validating country names, checking latitude and longitude values, grouping festivals by country and filtering by country or art form.

The page also includes an interactive map component. On web, the map uses Leaflet and React-Leaflet through components/FestivalMap.web.tsx. On native platforms, a separate map implementation is provided through components/FestivalMap.native.tsx.

This platform-specific component structure was necessary because web maps and native mobile maps use different libraries and rendering approaches. Separating them into different files allows the frontend to support both environments without forcing one platform's map implementation onto the other.

The Explore page therefore demonstrates one of the main strengths of the chosen frontend architecture: shared application logic can be combined with platform-specific components where required.

Festival Detail Page Architecture

The festival detail page is implemented using a dynamic route: app/festival/[id].tsx.

When a user selects a festival from the homepage, Explore page or map, the application navigates to the festival detail route using the festival's ID. The detail page reads the ID from the route parameters, validates it, and sends an API request to retrieve the matching festival.

The page displays a larger image, festival name, location, date, art form, website link and supporting description. If certain values are missing, the page uses fallback text such as "Date TBC" or avoids displaying empty information.

This page is important because it turns festival browsing into a more complete discovery experience. Instead of only showing users a list of festivals, the system gives each festival its own profile-style page.

Authentication Architecture

Authentication is implemented using a combination of login/signup screens, helper functions, platform-aware session storage and a global authentication context.

The authentication context is defined in lib/auth-context.tsx. It stores the current user session, whether the user is authenticated, whether the session is still loading, and functions for login and logout. Because this context wraps the whole application in the root layout, any page can access the user's authentication state.

Session persistence is handled in lib/auth-storage.ts. The storage method changes depending on platform. On web, the application uses localStorage. On native platforms, it uses Expo SecureStore. This is a practical architecture decision because each platform has different storage capabilities.

The login page sends the user's email and password to the backend login endpoint. If authentication is successful, the frontend extracts the returned token and saves the session through the authentication context. The signup page sends registration details to the backend register endpoint and validates form input before submission.

The profile page uses the authentication context to show different content depending on whether the user is logged in. Logged-out users are shown account options, while logged-in users see their account summary and can log out.

This authentication structure was important because the planning feature depends on knowing which user is currently logged in.

Planning and Wishlist Architecture

The planning page is implemented in app/(tabs)/plan.tsx and is one of the most important frontend features.

The page first checks the authentication state. If the user is not logged in, they are shown a clear message explaining that an account is required to use the wishlist feature. They are then given options to log in, create an account or browse festivals. This prevents unauthenticated users from trying to access features that require a backend user token.

When the user is logged in, the Plan page loads the user's wishlist from the backend using the stored bearer token. It also loads the festival list so that the user can search for festivals and add them to their wishlist.

The planning page is divided into tabs:

Wishlist allows users to search festivals from the database and add them to their saved list.

Month Selector allows users to filter their saved festivals by month. This was intentionally implemented as a month-based selector rather than a full day-by-day calendar because the available festival data is better suited to month-level planning.

Create with AI is currently a placeholder. The original intention was to connect this to an AI planner, but this was not completed due to time constraints and limited free AI options. The placeholder remains in the interface to demonstrate where the future feature would be integrated.

Share allows users to export a printable wishlist summary on the web version. The generated output includes festival names, countries, dates and website links.

The wishlist feature uses backend endpoints to add, retrieve and remove saved festivals. This means the saved data is connected to the user's account rather than being stored only in local frontend state.

Streaming Page Architecture

The streaming page is implemented in app/(tabs)/stream.tsx.

This page retrieves stream data from the backend using the /streams endpoint. The backend returns grouped data, so the frontend includes logic to flatten the response into stream cards that can be rendered consistently.

The stream page normalises platform names, validates stream URLs and creates automatic YouTube thumbnails where possible. This improves the visual presentation of the page without requiring every stream to manually include a thumbnail image.

Each stream card displays the stream title, festival name, category, description and platform label. Users can open the stream externally through the provided URL.

The streaming page supports the project objective of extending ArtoFest beyond a simple directory by giving users access to related festival media content.

Booking and Travel Ideas Page

The Book page is implemented in app/(tabs)/book.tsx.

This page is not a fully functional booking system. Instead, it acts as a prototype demonstration of how travel and accommodation planning could be presented in a future version of ArtoFest. It includes dropdown filters, mock travel/accommodation cards and buttons that route to a booking-unavailable page.

This was an important scope decision. The original project plan discussed travel and accommodation API integration as a future phase, but implementing real booking functionality would require external providers, payment systems, live availability, legal considerations and more time than was available within the project. Therefore, the frontend demonstrates the design direction without claiming to complete real ticket, hotel or transport booking.

The booking-unavailable page clearly explains that booking is not available in the prototype. This helps maintain user honesty and prevents the application from misleading users.

Responsive Design

Responsive design was a major part of the frontend architecture because the system needed to work across desktop and mobile screen sizes.

The frontend uses React Native styling, platform checks and screen width detection to adjust layouts. For example, the navigation bar changes between desktop and mobile layouts, the footer changes from a multi-column layout to a wrapped mobile layout, and several pages adjust between row-based and column-based layouts depending on screen width.

The Plan page also adapts its two-column wishlist layout depending on whether the screen is wide enough. On smaller screens, the panels stack vertically so that the content remains readable.

This responsive approach supports the project's cross-platform requirement and helps make the system usable on different devices without needing separate frontend projects.

Error Handling and User Feedback

The frontend includes several forms of error handling and user feedback.

Loading states are shown while data is being retrieved from the API. Error messages are displayed if festival data, wishlist data or stream content cannot be loaded. Forms provide validation feedback before submitting data to the backend. The planning page displays success and error messages when festivals are added to or removed from the wishlist.

The frontend also protects users from invalid actions. For example, unauthenticated users cannot use the wishlist tools, missing website links are handled safely, and PDF export shows an error if the wishlist is empty.

These user feedback mechanisms improve usability because users are not left guessing what has happened when a request is loading, fails, or succeeds.

Maintainability and Scalability

The frontend was structured with maintainability in mind. Shared logic is placed in the lib directory, reusable interface elements are placed in the components directory, and page-specific logic is kept inside the relevant route files.

This separation makes the application easier to extend. For example, if more festival fields are added to the backend, the festival helper type can be updated and reused

across the pages. If the backend API URL changes, it can be changed in one place. If the navigation needs to be updated, it can be changed in the top navigation component rather than across every page.

The architecture also supports future improvements:

Connecting the AI planner to a real AI service

Replacing mock booking content with real travel/accommodation API data

Improving native mobile PDF sharing

Further refining the admin/CMS functionality, including clearer permission feedback, improved validation, and additional client guidance for managing festival and stream records.

Deploying the frontend publicly once final testing is complete

UI/UX Design – Written by Favour Akuchie

The UI/UX design of ArtoFest was centred around creating a clean, modern, and user-friendly interface that enhances the festival discovery experience. The design prioritises clarity, ease of navigation, and visual appeal.

A consistent layout structure was implemented across all pages, including a top navigation bar and global footer. This ensures users can easily move between key sections such as Home, Explore, Plan, Book, and Stream without confusion.

The homepage was designed as the primary entry point, featuring a strong visual hierarchy with a hero section and clearly defined search filters. Dropdown filters for country, month, and art form allow users to quickly refine their search, improving usability and reducing cognitive load.

The Explore page adopts both grid and map-based layouts to support different browsing preferences. The grid view allows users to scan multiple festivals quickly, while the map view provides a geographical perspective, enhancing spatial awareness.

Festival cards were designed to display essential information at a glance, including images, location, and category. This ensures that users can quickly decide which festivals to explore further.

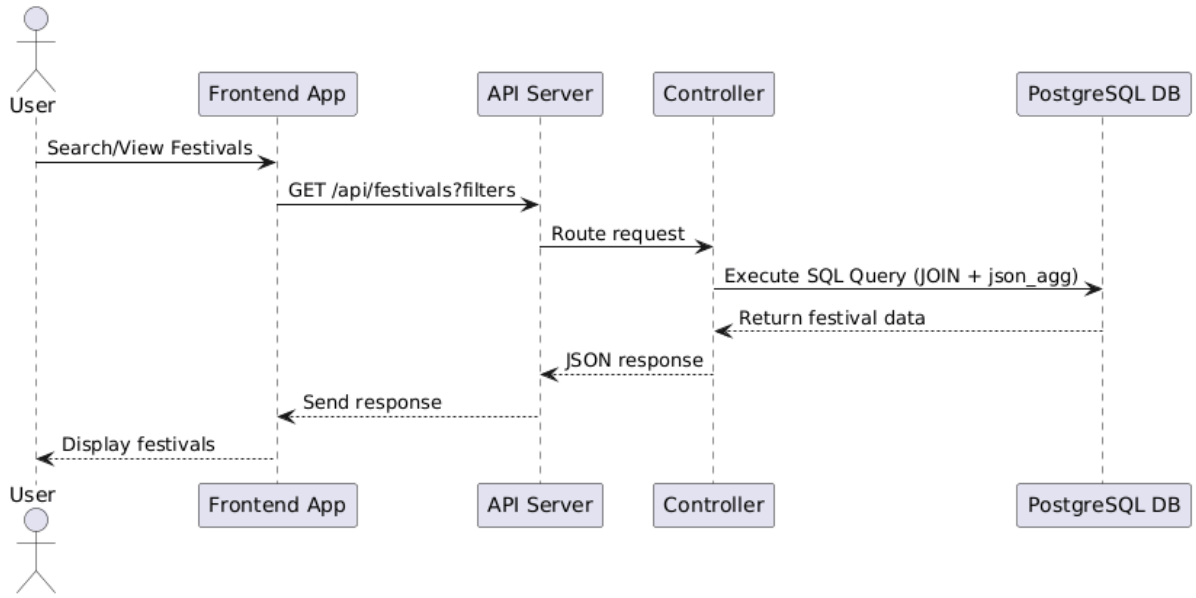
The overall colour scheme and typography were chosen to maintain readability while creating a visually engaging interface. Spacing, alignment, and responsive layouts were carefully considered to ensure the system performs well across different screen sizes.

User experience was further enhanced through:

- Clear feedback messages (loading, errors, success states)
- Simple and intuitive navigation
- Responsive design for mobile and desktop
- Minimal clutter to avoid overwhelming users

The final design supports both first-time users and returning users by balancing simplicity with functionality.

Sequence Diagram - Written by Idowu Adeleke



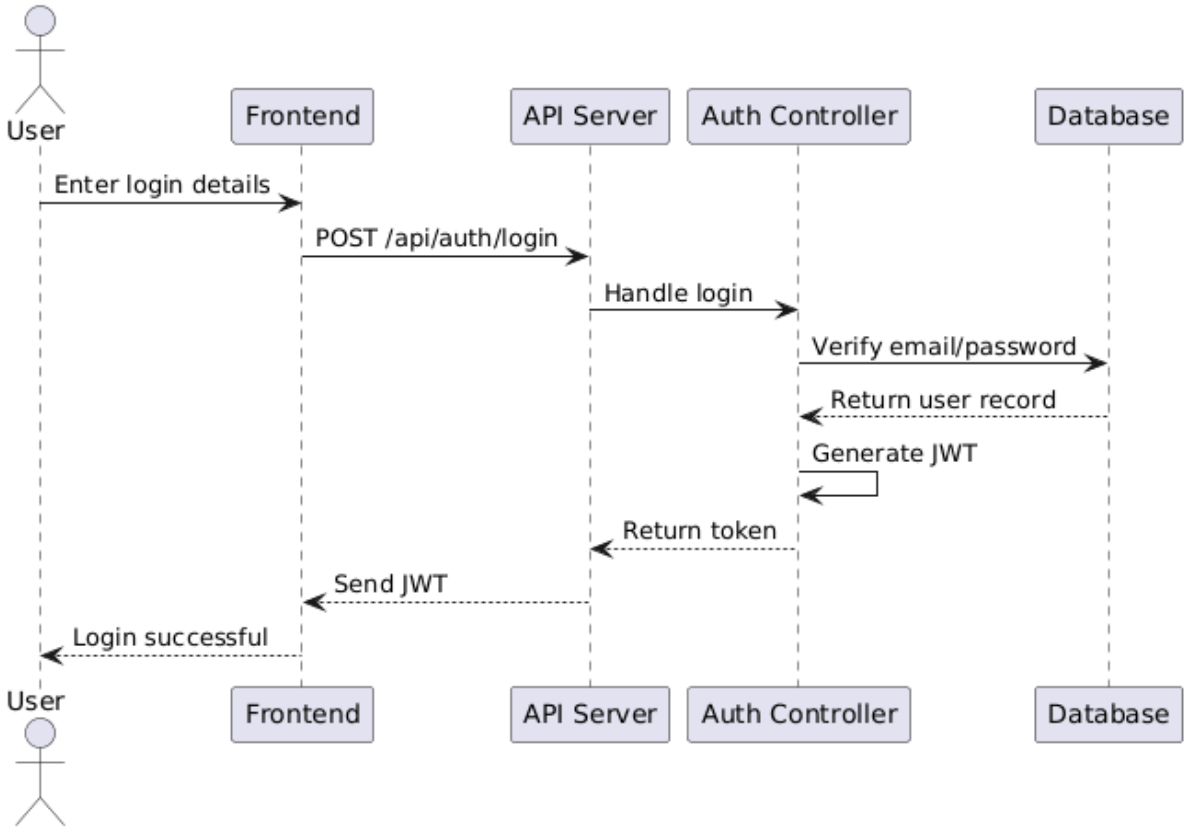
The sequence diagram illustrates the interaction between system components during a festival retrieval request.

The process begins when the user initiates a search or requests festival data through the frontend interface. The frontend sends a GET request to the API endpoint `/api/festivals`, optionally including filtering parameters such as country, genre, or search keywords.

Upon receiving the request, the API routes it to the appropriate controller. The controller then executes a structured SQL query that joins multiple tables, including festivals, genres, and festival_images. The query also uses JSON aggregation to group related images into an array format.

The database returns the processed data to the controller, which formats it into a JSON response. This response is sent back through the API to the frontend, where it is rendered and displayed to the user.

This sequence ensures efficient data retrieval while minimizing the number of API calls required by the frontend.



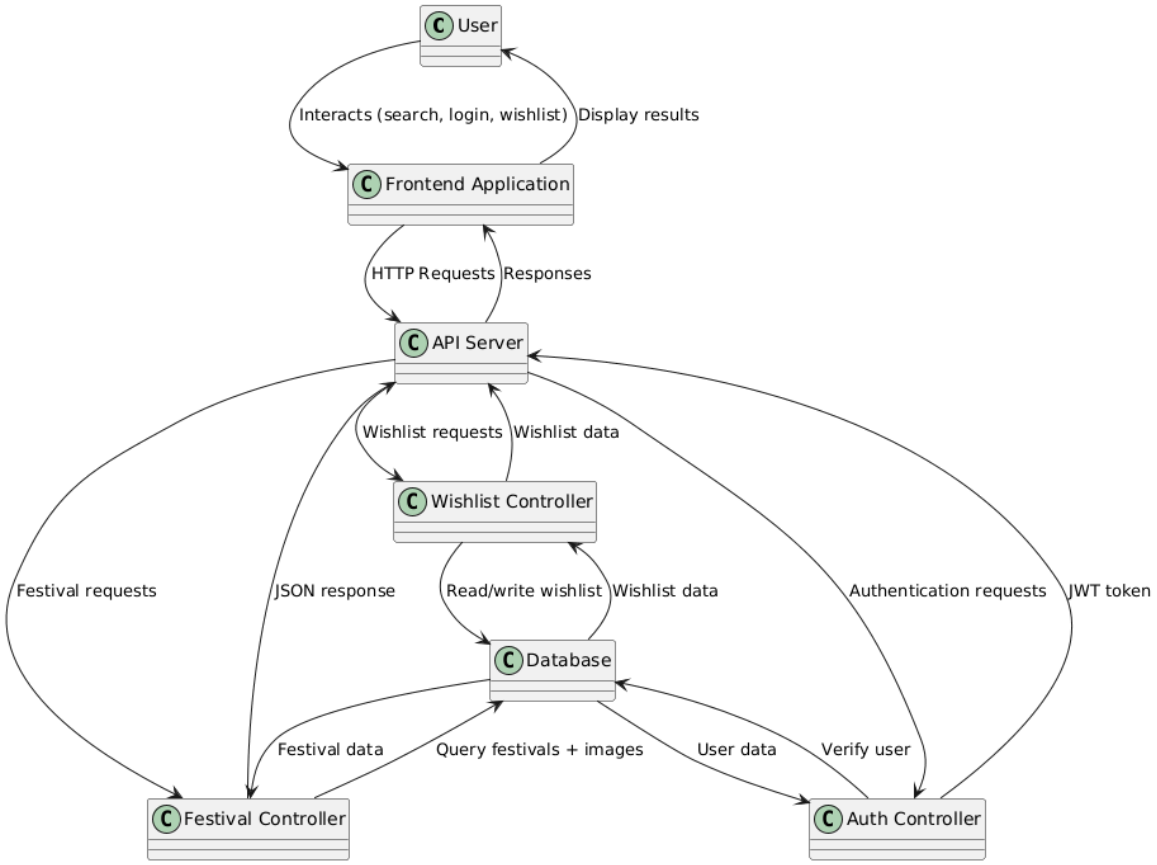
This sequence diagram represents the authentication process within the system.

The user submits login credentials via the frontend interface, which sends a POST request to the `/api/auth/login` endpoint. The API forwards the request to the authentication controller.

The controller queries the database to verify the user's credentials. If valid, a JSON Web Token (JWT) is generated and returned to the client. The frontend stores this token and uses it to access protected endpoints such as the wishlist.

This approach ensures secure and stateless authentication.

Data Flow Diagram - Written by Idowu Adeleke



The Data Flow Diagram (DFD) illustrates how data moves through the ArtOfest system.

The user interacts with the frontend application to perform actions such as searching for festivals, logging in, or managing a wishlist. The frontend sends HTTP requests to the API server, which acts as the central processing unit of the system.

The API routes requests to specific controllers depending on the operation:

- Festival Controller handles festival data retrieval
- Authentication Controller manages user login and registration
- Wishlist Controller manages user-specific saved festivals

Each controller communicates with the PostgreSQL database to either retrieve or store data. The database processes queries and returns structured results to the controllers.

The controllers then format the data into JSON responses, which are sent back through the API to the frontend. Finally, the frontend displays the processed information to the user.

This architecture ensures a clear separation of concerns and efficient data handling across the system.

Design Justification

The combination of sequence diagrams and DFDs provides a comprehensive understanding of the system's behavior.

- **Sequence diagrams** highlight interaction timing and flow between components
- **DFDs** illustrate how data is processed and transferred across the system

This dual approach improves system clarity, maintainability, and scalability.

State Diagram - Written by Jacob Askew

The state diagram represents the main user-facing states within the ArtoFest frontend application. It focuses on how a user moves through the system depending on whether they are logged in or logged out and how this affects access to personalised functionality such as the wishlist and planning tools.

The frontend application has two broad user states: unauthenticated and authenticated. Unauthenticated users can still browse the homepage, search festivals, use the Explore page, view festival detail pages, open the Stream page,and view the prototype Book page. However, they cannot access the personalised wishlist features because these require a valid user session and bearer token.

When a user registers or logs in successfully, the frontend stores the authenticated session using the platform-appropriate storage method. On web, this is handled through localStorage. On native platforms, Expo SecureStore is used. Once the session is stored, the user enters the authenticated state and can access account-specific features such as viewing their profile, adding festivals to their wishlist, removing saved festivals, filtering saved festivals by month, and generating a printable plan summary.

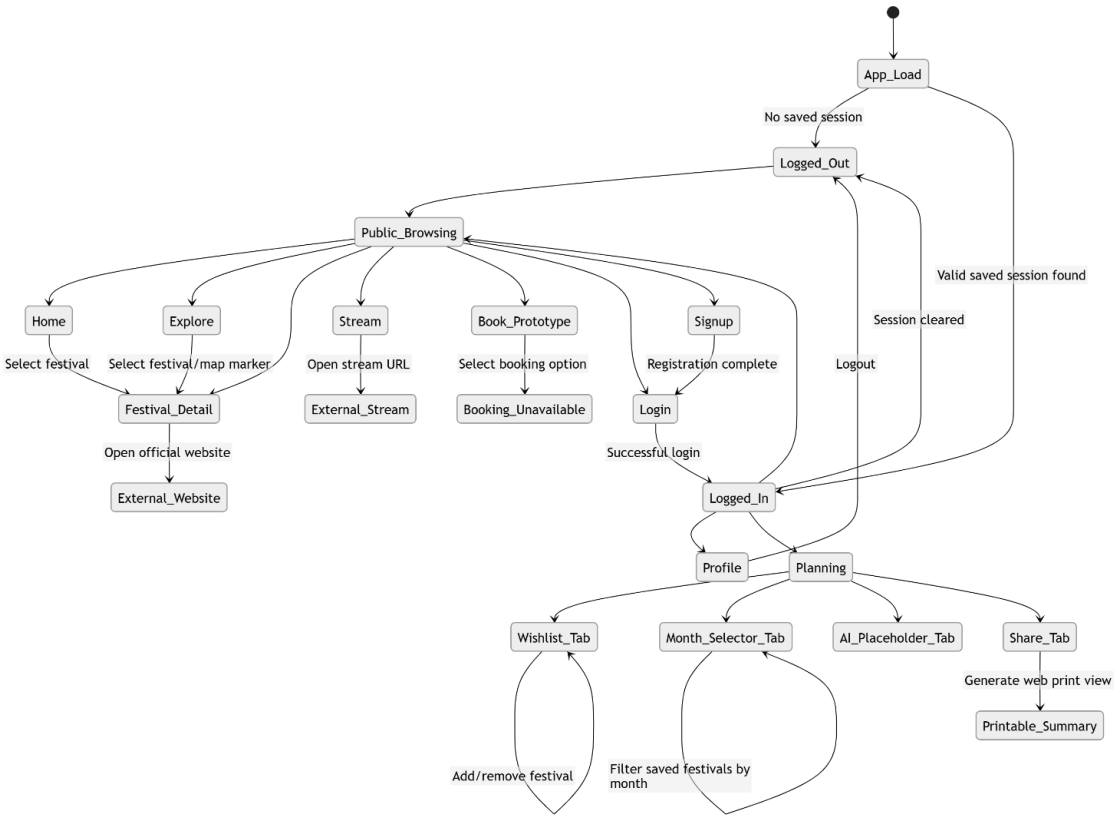
The Plan page demonstrates the clearest state-based behaviour in the frontend. If the user is logged out, the page displays an account-required message and provides routes to Login, Signup, and Browse Festivals. If the user is logged in, the page loads the user's wishlist from the backend using the saved authentication token. The page then allows the user to move between the Wishlist, Month Selector, Create with AI placeholder, and Share tabs.

The state diagram also shows that some features are available regardless of authentication. For example, festival browsing, map exploration, stream viewing and opening external festival websites are public-facing features. This design supports

casual users who want to discover festivals without immediately creating an account, while still encouraging account creation for personalised planning.

The main states in the frontend are:

- Initial App Load
  - The application checks whether a saved session exists.
- Logged-Out State
  - The user can browse public pages but cannot use personalised planning features.
- Browsing State
  - The user can use Home, Explore, festival detail pages, Stream, Book, About and FAQ pages.
- Authentication State
  - The user can register or log in.
- Logged-In State
  - The user has a valid session and can access profile and planning features.
- Planning State
  - The user can view saved festivals, add/remove wishlist items, filter by month and generate a shareable/printable summary.
- Logout State
  - The saved session is cleared and the user returns to the logged-out state.



The state diagram shows that ArtoFest is not a single linear application. Instead, users can move between public discovery features and account-based planning features depending on their authentication state. This was an important frontend design decision because it keeps the platform accessible to new users while still protecting personalised wishlist data behind login.

The diagram also shows the difference between fully implemented features and prototype/future features. The booking page routes users to a booking-unavailable page rather than completing a real booking. Similarly, the AI planner tab exists as a placeholder because the intended AI planning feature was not completed due to time constraints and limited free AI service options. Including these states accurately reflects the finished MVP and avoids presenting unfinished functionality as complete.

Flowchart - Written by Jacob Askew

The flowchart illustrates the main frontend user journey for discovering a festival and optionally saving it to a personal wishlist. This process was selected because it represents the core purpose of ArtoFest: helping users discover festivals, view relevant details and plan around festivals they are interested in.

The flow begins when the user opens the application. From the homepage, the user can browse featured content or use the search filters. The search filters allow the user to filter festivals by country, month and art form. Once the user performs a search, the frontend compares the selected filters against the festival data retrieved from the backend API and displays matching festival cards.

The user can then select a festival card to open the festival detail page. This page retrieves the selected festival by ID and displays additional information such as the image, location, date, art form, website and description. From this point, the user can either open the official festival website or continue using ArtoFest.

If the user wants to save a festival, they must use the Plan page. The flowchart shows a decision point where the frontend checks whether the user is logged in. If the user is not authenticated, the Plan page displays a message explaining that an account is required and directs the user to log in or sign up. If the user is authenticated, the frontend loads their wishlist using the saved bearer token and allows the selected festival to be added.

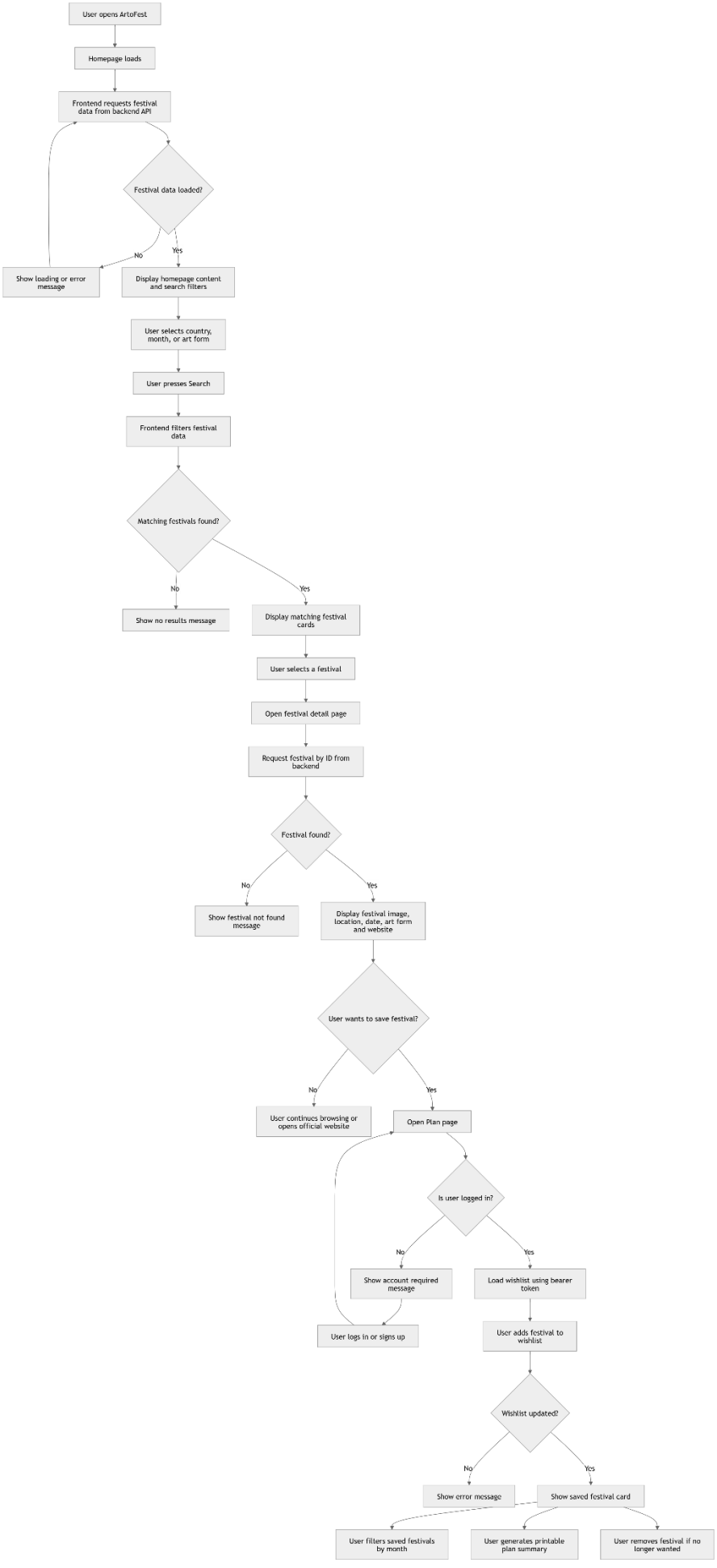
The flowchart also includes the month selector and share functions. Once festivals have been saved, the user can filter saved festivals by month or generate a printable plan summary on the web version.

The flowchart shows how the frontend handles the main user journey from discovery to planning. It also demonstrates how the frontend manages different outcomes,

including API loading errors, no search results, invalid festival IDs, logged-out access restrictions and wishlist update failures.

This flow reflects the way the application was implemented as an MVP. Public festival discovery is kept open to all users, while personalised planning requires authentication. This supports usability because users can explore the platform before creating an account, but it also protects user-specific wishlist data by requiring a valid login session before any saved festival operations can be performed.

The flowchart also shows the importance of frontend error handling. The application does not assume that API requests will always succeed. Instead, it provides loading states, error messages and fallback behaviour so that users receive clear feedback when something goes wrong.

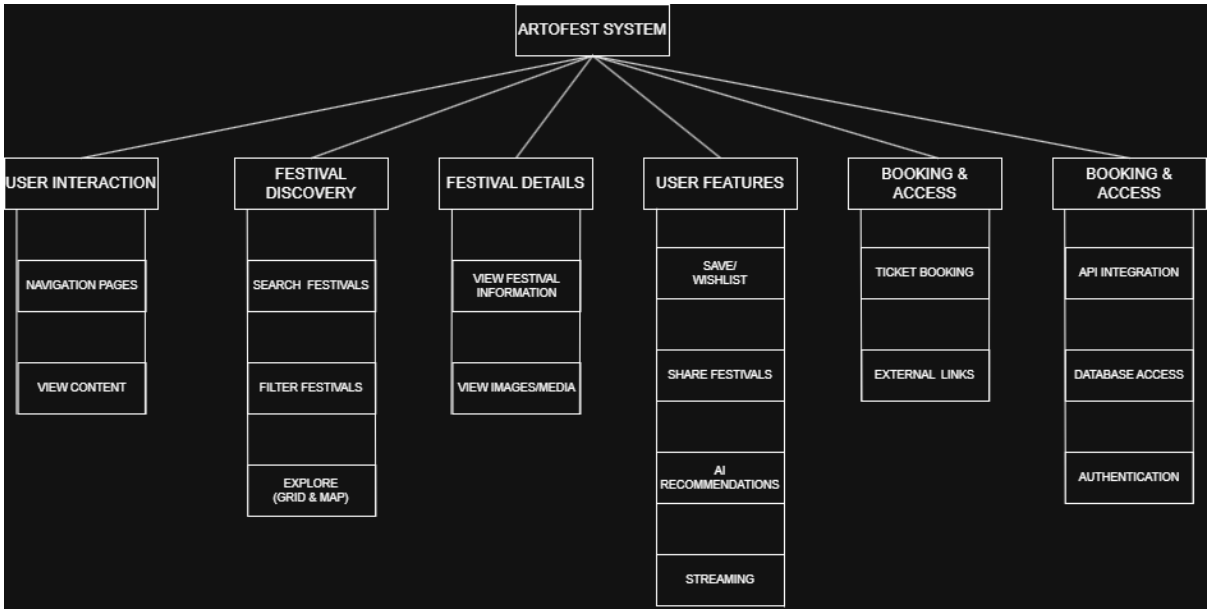

User opens ArtoFest
Homepage loads
Frontend requests festival data from backend API
Festival data loaded?
No
Yes
Show loading or error message
Display homepage content and search filters
User selects country, month, or art form
User presses Search
Frontend filters festival data
Matching festivals found?
No
Yes
Show no results message
Display matching festival cards
User selects a festival
Open festival detail page
Request festival by ID from backend
Festival found?
No
Yes
Show festival not found message
Display festival image, location, date, art form and website
User wants to save festival?
No
Yes
User continues browsing or opens official website
Open Plan page
Is user logged in?
No
Yes
Show account required message
Load wishlist using bearer token
User logs in or signs up
User adds festival to wishlist
Wishlist updated?
No
Yes
Show error message
Show saved festival card
User filters saved festivals by month
User generates printable plan summary
User removes festival if no longer wanted

HIPO Chart - Written by Paul Oko-Jaja

The HIPO (Hierarchy plus Input-Process-Output) chart is used to represent the structure and functionality of the artofest.com system at a high level. It provides a clear breakdown of the system into its main components and illustrates how user interactions are processed to produce outputs.

The hierarchy diagram outlines the main modules of the system, including user interaction, festival discovery, festival details, user features, booking functionality, and system services. This structure demonstrates how the system is organised and how different components interact to deliver core functionality.

To further illustrate system behaviour, Input-Process-Output (IPO) tables were created for key functions within the system.



Festival Search & Filter

input	process	output
User-selected filters (e.g. country, genre)	System processes filter criteria and queries the database for matching festivals	Filtered list of festivals displayed

Explore Page (Grid & Map)

input	process	output
Festival data retrieved from database/API	Data is processed and rendered into grid and map views	Visual display of festivals in grid and map format

Festival Details Page

input	process	output
Selected festival	System retrieves detailed information and associated media	Full festival details displayed to the user

Festival Details Page

input	process	output
User interaction (clicks/navigation)	Routing system processes request and loads appropriate page	User is navigated to selected page

UML Diagram - Written by Favour Akuchie

The UML diagram provides a high-level representation of the system structure and interactions between different components within ArtoFest.

The diagram illustrates the relationship between key entities such as:

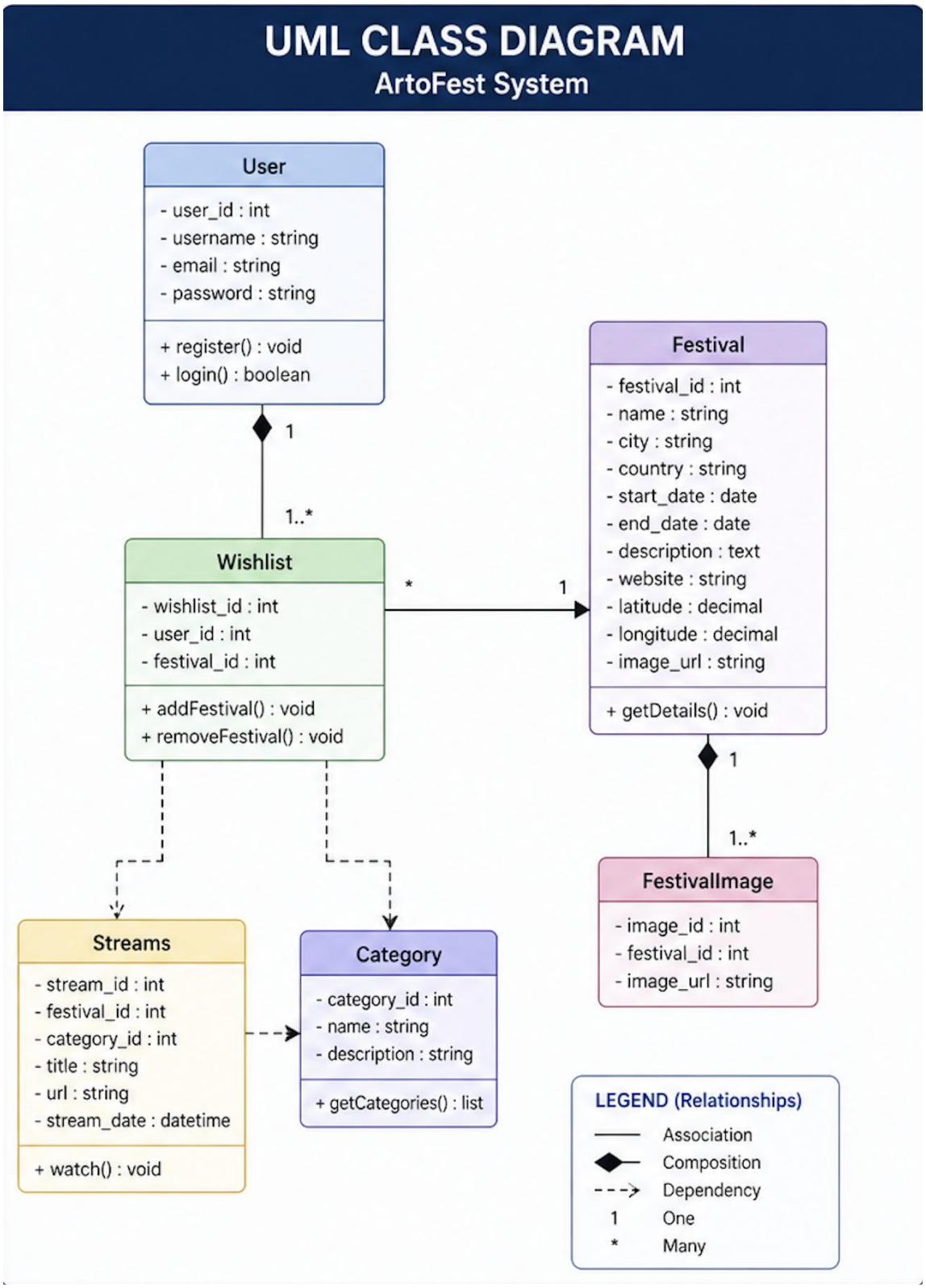
- User
- Festival
- Wishlist
- Streams

Users interact with the system through actions such as searching for festivals, viewing details, and managing their wishlist. These interactions are connected to backend processes that handle data retrieval, authentication, and storage.

The UML diagram highlights:

- The connection between users and their personalised data (wishlist)

- The relationship between festivals and associated media (images and streams)
- The flow of data between frontend and backend components



Frontend Code Snippets – Written by Jacob Askew

_layout.tsx M

app > (tabs) > _layout.tsx > ...

```tsx
import { Slot } from "expo-router";
import React from "react";
import { SafeAreaView } from "react-native-safe-area-context";

export default function TabLayout() {
  return (
    <SafeAreaView style={{ flex: 1 }} edges={["bottom", "left", "right"]}>
      <Slot />
    </SafeAreaView>
  );
}
```

This snippet shows the root layout of the ArtoFest frontend. The application is wrapped in a global authentication provider so that login state can be accessed across all screens. The custom top navigation is also placed at the root level, meaning it appears consistently throughout the application without needing to be repeated manually on each page. Expo Router's Slot component renders the active route, allowing the application to behave like a multi-page web/mobile app while still using a shared React Native codebase.

api.ts M

lib > api.ts > ...

```ts
export const API_BASE_URL = "https://artofest-api-yc0q.onrender.com/api";
```

festivals.ts U

lib > festivals.ts > normaliseString

```ts
export function safeWebUrl(url: string) {
  const trimmed = url.trim();
  if (!trimmed) return "";
  if (!/^https?:\/\//i.test(trimmed)) return `https://${trimmed}`;
  return trimmed;
}
```

```ts
export function fetchFestivals() {
  return fetch(`${API_BASE_URL}/festivals`);
}

export function fetchFestivalById(id: number) {
  return fetch(`${API_BASE_URL}/festivals/${id}`);
}
```

```ts
export function normaliseMonthText(value: unknown) {
  const text = String(value ?? "").trim();
  if (!text) return "";

  const lower = text.toLowerCase();

  const monthMap: Record<string, string> = {
    january: "January",
    jan: "January",
    february: "February",
    feb: "February",
    march: "March",
    mar: "March",
    april: "April",
    apr: "April",
    may: "May",
    june: "June",
    jun: "June",
    july: "July",
    jul: "July",
    august: "August",
    aug: "August",
    september: "September",
    sep: "September",
    sept: "September",
    october: "October",
    oct: "October",
    november: "November",
    nov: "November",
    december: "December",
    dec: "December",
  };

  return monthMap[lower] ?? "";
}
```

These snippets above show how the frontend centralises backend communication and festival data handling. The API base URL is stored in one file so that the backend address can be updated easily if the API changes. The festival helper file contains reusable functions for retrieving festival data, formatting links safely, normalising month values, and requesting specific festival records by ID. This avoids repeating the same logic across multiple pages and makes the frontend easier to maintain.

index.tsx M

app > (tabs) > index.tsx > HomeSearchScreen > useEffect() callback > loadFestivals

```tsx
export default function HomeSearchScreen() {
  useEffect(() => {
    async function loadFestivals() {
      try {
        setLoading(true);
        setError("");

        const response = await fetchFestivals();

        if (!response.ok) {
          throw new Error(`Request failed with status ${response.status}`);
        }

        const data = await response.json();

        if (!Array.isArray(data)) {
          throw new Error("API did not return an array");
        }

        const cleanedFestivals = data
          .map((item: FestivalRecord) => ({
            id: toNumericId(item.id),
            name: normaliseString(item.name),
            city: normaliseString(item.city),
            country: normaliseString(item.country),
            image_url: normaliseString(item.image_url),
            website: normaliseString(item.website),
            start_date: item.start_date ?? null,
            end_date: item.end_date ?? null,
            month_text: normaliseString(item.month_text),
            art_form: normaliseString(item.art_form),
            description: normaliseString(item.description),
            latitude: item.latitude ?? null,
            longitude: item.longitude ?? null,
          }))
          .filter(
            (festival: FestivalRecord) =>
              typeof festival.id === "number" && Number.isFinite(festival.id)
          );

        setFestivals(cleanedFestivals);
      } catch (err) {
        console.error("Failed to load festivals:", err);
        setError("Failed to load festivals from the server.");
```

```
const results = useMemo(() => {
  if (!hasSearched) return [];
  if (!atLeastOneParamSelected) return [];

  return festivals.filter((festival) => {
    const country = normaliseString(festival.country).toLowerCase();
    const month = normaliseMonth(
      festival.month_text,
      festival.start_date
    ).toLowerCase();
    const artForm = normaliseString(festival.art_form).toLowerCase();

    if (selectedCountry && country !== selectedCountry.trim().toLowerCase()) {
      return false;
    }

    if (selectedMonth && month !== selectedMonth.trim().toLowerCase()) {
      return false;
    }

    if (selectedArtForm && artForm !== selectedArtForm.trim().toLowerCase()) {
      return false;
    }

    return true;
  });
```

```
function openFestivalDetail(festival: FestivalRecord) {
  if (typeof festival.id !== "number" || !Number.isFinite(festival.id)) {
    console.log("Festival has invalid id:", festival);
    return;
  }

  router.push({
    pathname: "/festival/[id]",
    params: { id: String(festival.id) },
  });
}
```

These snippets demonstrate how the homepage loads festival data from the backend and prepares it for user-facing search. The frontend checks that the response is valid, normalises important fields, removes invalid records, and stores the cleaned festival list in state. The search results are calculated using useMemo, which means results are only recalculated when the festival data or selected filters change. The filter system supports country, month and art form, allowing users to narrow down festivals dynamically. The final part of the snippet shows route-based navigation, where selecting a festival opens its individual detail page using the festival ID.

```tsx
export default function FestivalDetailPage() {
  const params = useLocalSearchParams<{ id?: string | string[] }>();

  const festivalId = useMemo(() => {
    const raw = Array.isArray(params.id) ? params.id[0] : params.id;
    return toNumericId(raw);
  }, [params.id]);
```

```tsx
useEffect(() => {
  async function loadFestival() {
    if (typeof festivalId !== "number" || !Number.isFinite(festivalId)) {
      setError("Invalid festival ID.");
      setLoading(false);
      return;
    }

    try {
      setLoading(true);
      setError("");

      const response = await fetchFestivalById(festivalId);

      if (!response.ok) {
        throw new Error(`Request failed with status ${response.status}`);
      }

      const data = (await response.json()) as FestivalRecord;

      setFestival({
        id: toNumericId(data.id),
        name: normaliseString(data.name),
        city: normaliseString(data.city),
        country: normaliseString(data.country),
        image_url: normaliseString(data.image_url),
        website: normaliseString(data.website),
        start_date: data.start_date ?? null,
        end_date: data.end_date ?? null,
        month_text: normaliseString(data.month_text),
        art_form: normaliseString(data.art_form),
        description: normaliseString(data.description),
        latitude: data.latitude ?? null,
        longitude: data.longitude ?? null,
      });
    } catch (err) {
      console.error("Failed to load festival details:", err);
      setError("Failed to load this festival.");
    } finally {
      setLoading(false);
    }
  }

  loadFestival();
}, [festivalId]);
```

```
const descriptionText =
  normaliseString(festival?.description) ||
  "A full festival description will appear here once it has been added to the database.";
```

These snippets show the dynamic festival detail page. When a user selects a festival, the frontend reads the festival ID from the route, validates it and requests the matching festival from the backend. The returned data is normalised before being displayed, which reduces the risk of broken UI caused by missing or inconsistent API values. Loading and error states are included so that users receive clear feedback if the festival is still loading or cannot be retrieved.

```
try {
  const response = await fetch(`${API_BASE_URL}/auth/login`, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify({
      email: cleanedEmail,
      password,
    }),
  });
```

```
    const token = extractToken(payload);

    if (!token) {
      setBanner({
        tone: "error",
        message:
          "The login response succeeded but no token was returned. The backend response needs checking.",
      });
      return;
    }

    const username = extractUsername(payload);

    await login({
      token,
      email: cleanedEmail,
      username,
    });

    setPassword("");
    setFieldErrors({});
    setShowPassword(false);

    router.replace("/profile");
  } catch {
```

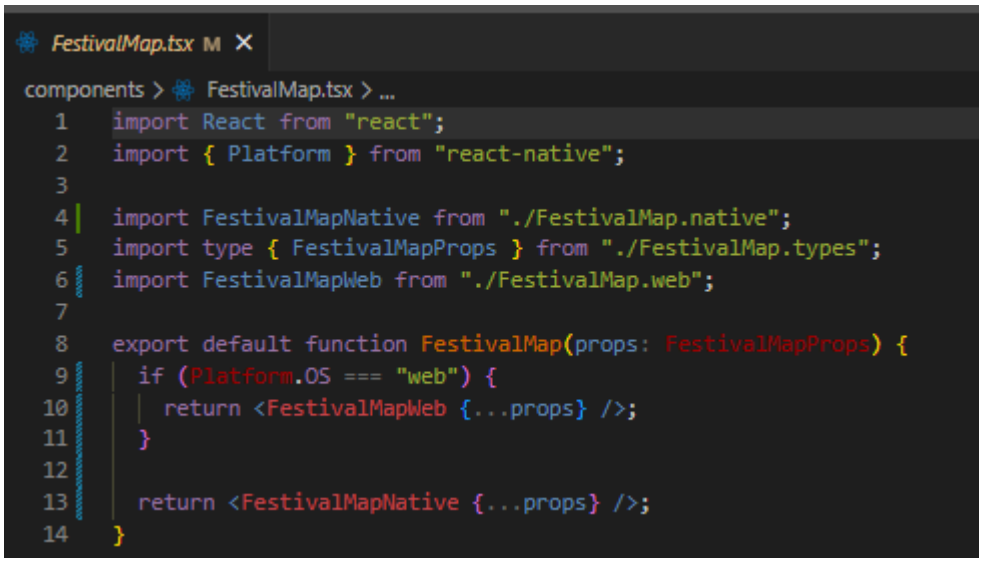
```
async function login(payload: LoginPayload) {
  const nextSession: AuthSession = {
    token: payload.token,
    email: payload.email,
    username: payload.username ?? null,
    loggedInAt: new Date().toISOString(),
  };

  await saveAuthSession(JSON.stringify(nextSession));
  setSession(nextSession);
}

async function logout() {
  await clearAuthSession();
  setSession(null);
}
```

```
export async function saveAuthSession(value: string) {
  if (Platform.OS === "web") {
    if (canUseWebStorage()) {
      window.localStorage.setItem(AUTH_STORAGE_KEY, value);
    }
    return;
  }

  await SecureStore.setItemAsync(AUTH_STORAGE_KEY, value);
}
```

These snippets demonstrate the frontend authentication flow. The login page sends the user's email and password to the backend authentication endpoint and expects a token in response. If a token is returned, the frontend creates an authentication session containing the token, email, username and login timestamp. This session is saved through the authentication context so that other pages can check whether the user is logged in. The storage layer uses localStorage on web and Expo SecureStore on native platforms, allowing the same codebase to support both browser and mobile environments.

```
async function loadWishlist() {
  if (!session?.token) return;

  try {
    setWishlistLoading(true);
    setError("");

    const response = await fetch(`${API_BASE_URL}/wishlist`, {
      headers: getAuthHeaders(session.token),
    });

    if (!response.ok) {
      throw new Error(`Wishlist request failed with status ${response.status}`);
    }

    const data = await response.json();

    if (!Array.isArray(data)) {
      throw new Error("Wishlist API did not return an array.");
    }

    const cleaned = data
      .map(cleanWishlistItem)
      .filter((item): item is WishlistItem => item !== null);

    setWishlist(cleaned);
  } catch (err) {
    console.error("Failed to load wishlist:", err);
    setError(
      "Could not load your wishlist. Check that the backend wishlist endpoint accepts the login token."
    );
  } finally {
    setWishlistLoading(false);
  }
}
```

This snippet shows how the Plan page loads the logged-in user's wishlist from the backend. The function first checks whether a session token exists. If no token is available, the request is not made. When a token is present, the frontend sends a request to the /wishlist endpoint using authenticated headers. The returned data is checked to confirm that it is an array, then each item is cleaned using cleanWishlistItem before being stored in the page state. This prevents invalid wishlist records from being displayed and ensures that the planning page uses account-linked backend data rather than static or local-only data. Loading and error states are also handled so the user receives feedback if the wishlist cannot be retrieved.



This snippet shows how the ArtoFest frontend supports platform-specific map rendering. The shared FestivalMap component checks the current platform using React Native's Platform.OS. If the application is running on web, it renders the web map implementation from FestivalMap.web.tsx. If the application is running on a native mobile platform, it renders the native implementation from FestivalMap.native.tsx. This design allows the frontend to keep one shared map component interface while still using the correct map library for each platform.

This was necessary because web and native map libraries work differently. The web version uses Leaflet and React-Leaflet to display festival locations in the browser, while the native version uses react-native-maps with mobile map markers and callouts. By separating the implementations into web and native files, the application avoids compatibility issues and supports the project objective of using a shared React Native/Expo codebase across web and mobile environments.

GitHub Repository

https://github.com/Plymouth-University/comp2003-2025-2026-team-31

Unit Testing - Written by Idowu Adeleke

Overview of Testing Strategy

Testing and validation were critical in ensuring that the backend of the Artofest system operates reliably, securely, and efficiently. The testing approach focused on validating API endpoints, database queries, and error handling mechanisms.

The backend was tested using a combination of:

- **Manual API testing (Postman)**
- **Validation of SQL queries**
- **Error handling verification**
- **Authentication testing using JWT**

This ensured that all endpoints function correctly under different scenarios and inputs.

API Testing Using Postman

Postman was used extensively to test all API endpoints during development.

Tested Endpoints Include:

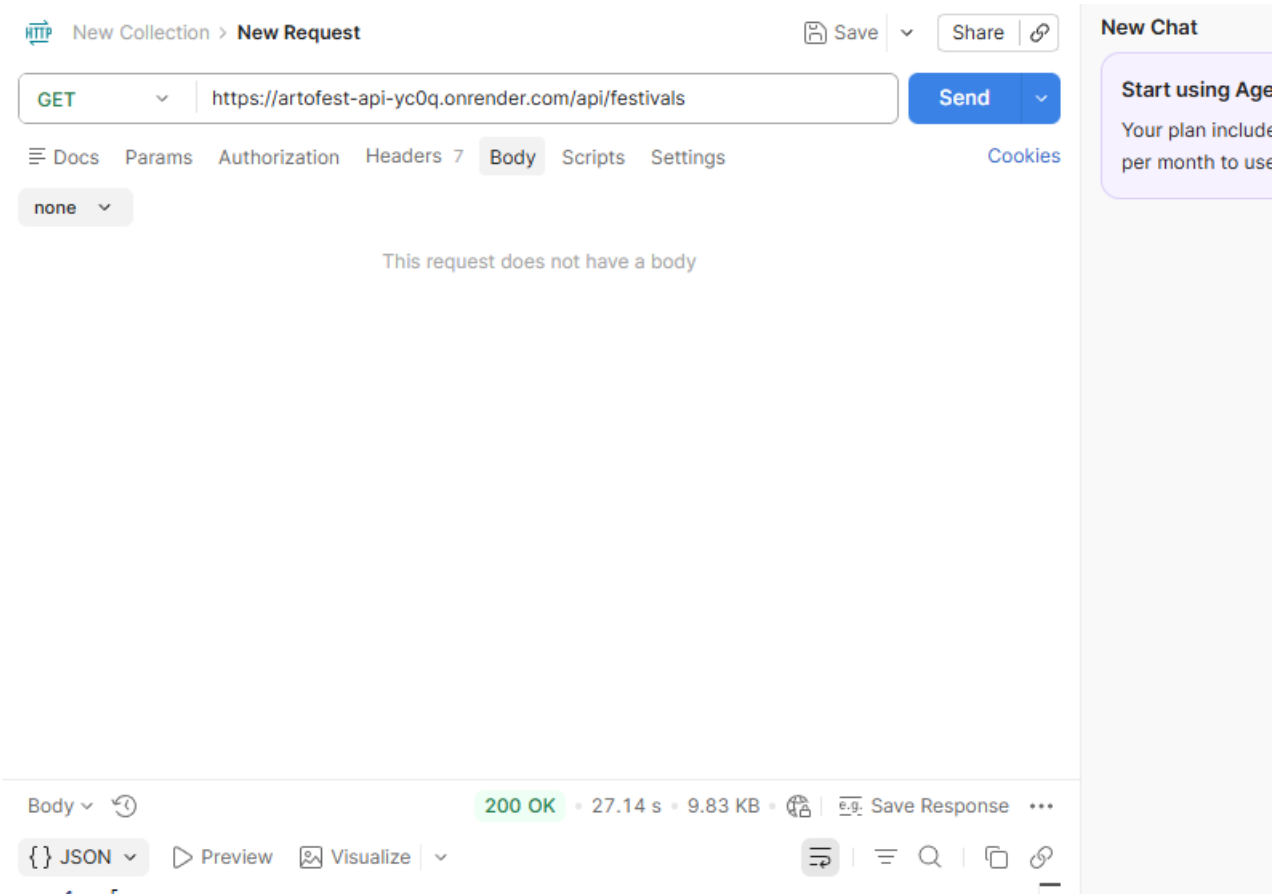
- `GET /api/festivals`
- `GET /api/festivals/:id`
- `POST /api/auth/register`
- `POST /api/auth/login`
- `GET /api/wishlist`
- `POST /api/wishlist`
- `DELETE /api/wishlist/:festivalId`
- `GET /api/streams`

Test Cases Performed

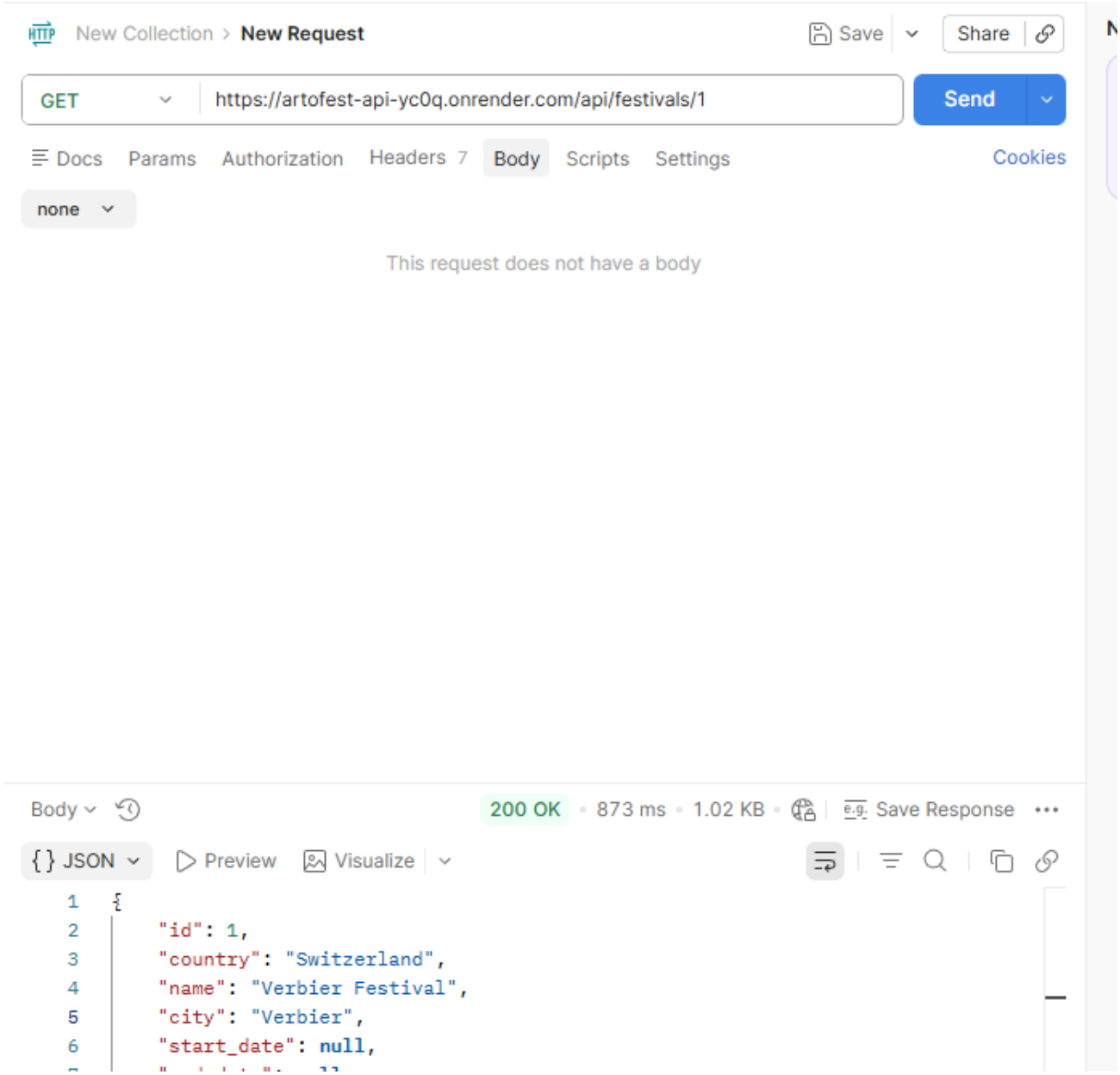
Test Case	Description	Expected Result
Valid request	Correct parameters provided	200 OK response
Invalid ID	Non-existent festival ID	404 Not Found

Missing fields	Missing required input	400 Bad Request
Unauthorized access	No JWT token	401 Unauthorized
Server error	Database failure simulation	500 Server Error

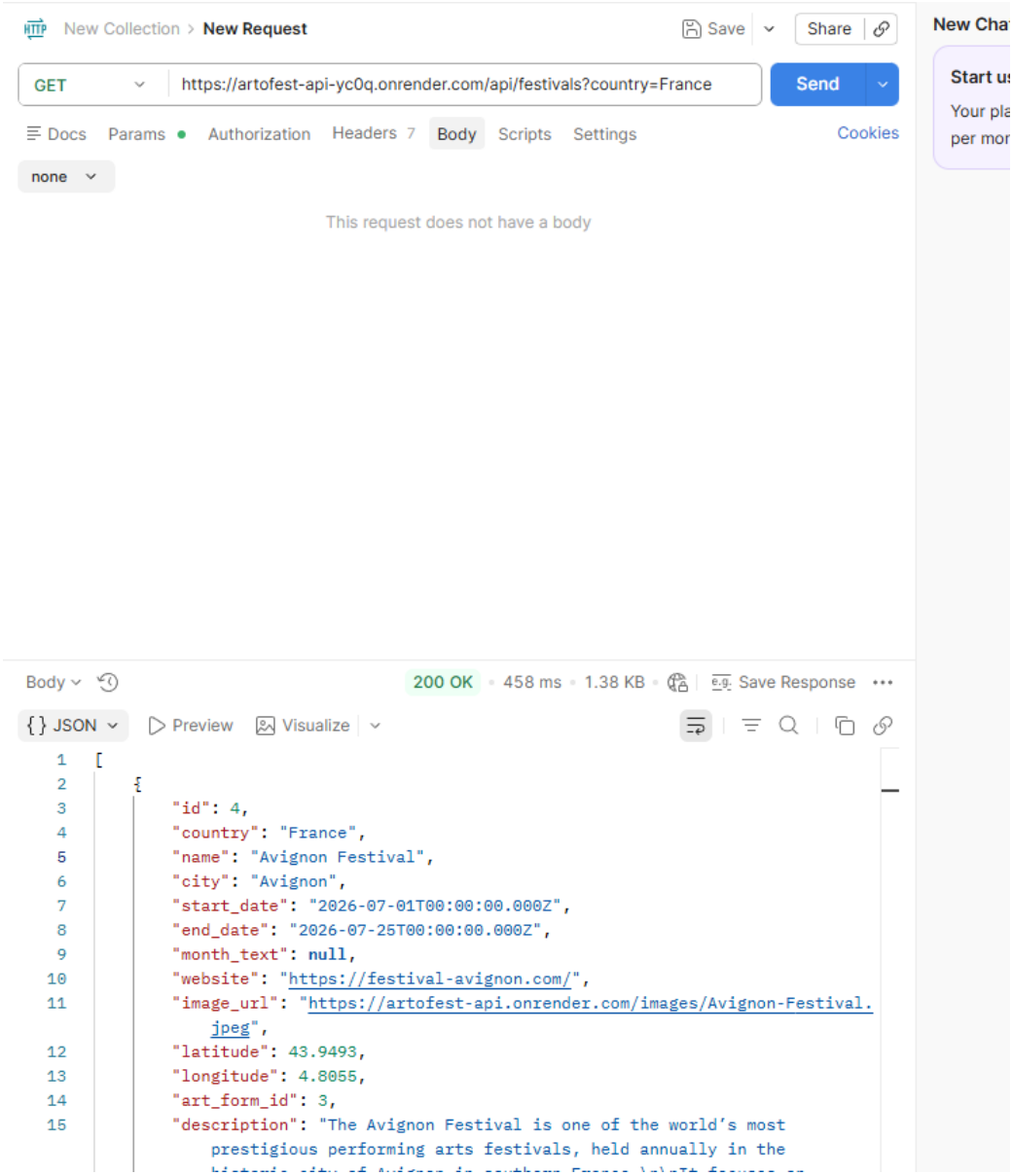
TEST 1: Testing endpoint of get all festivals



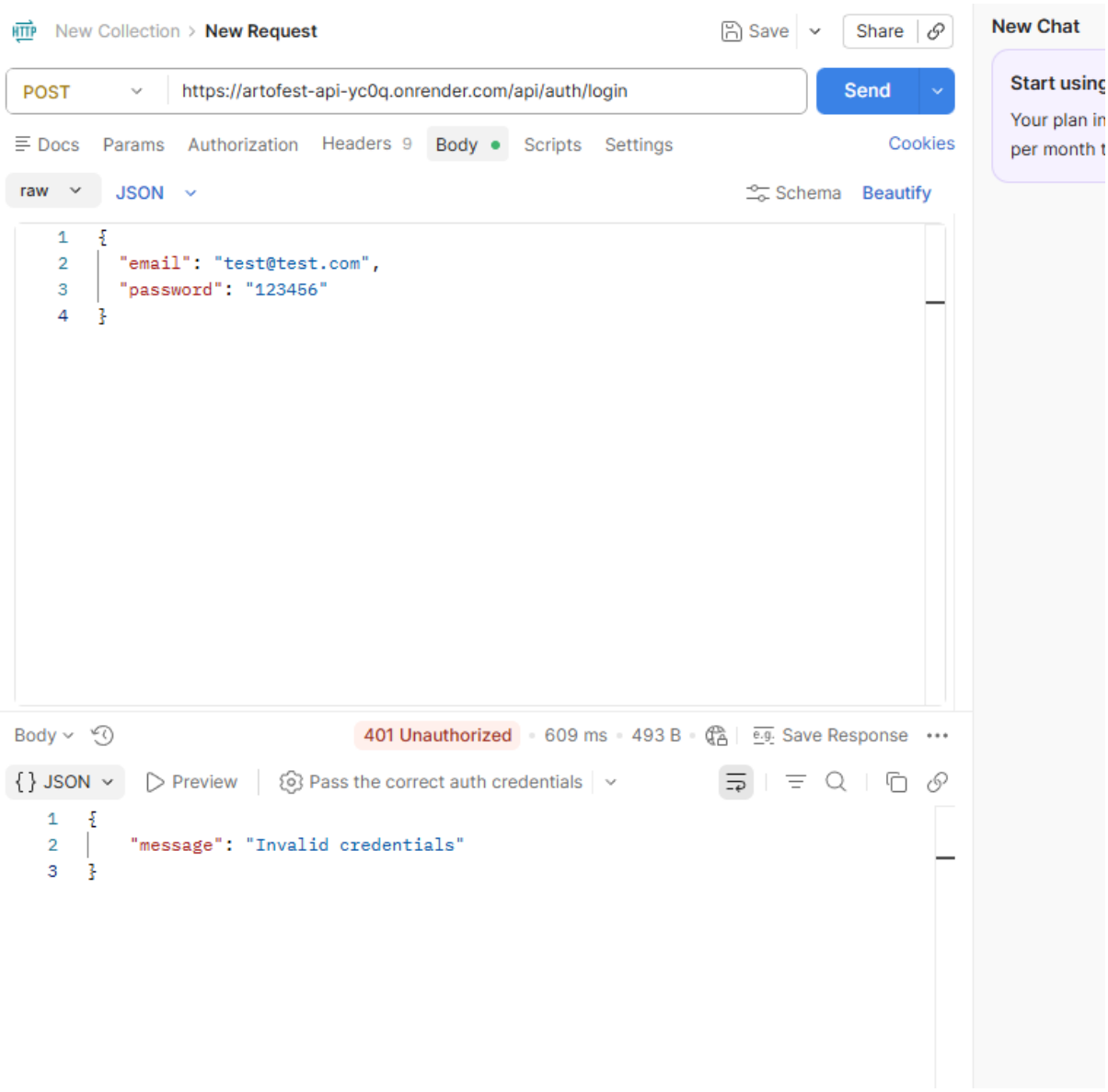
TEST 2: Testing endpoint of getfestivalbyId



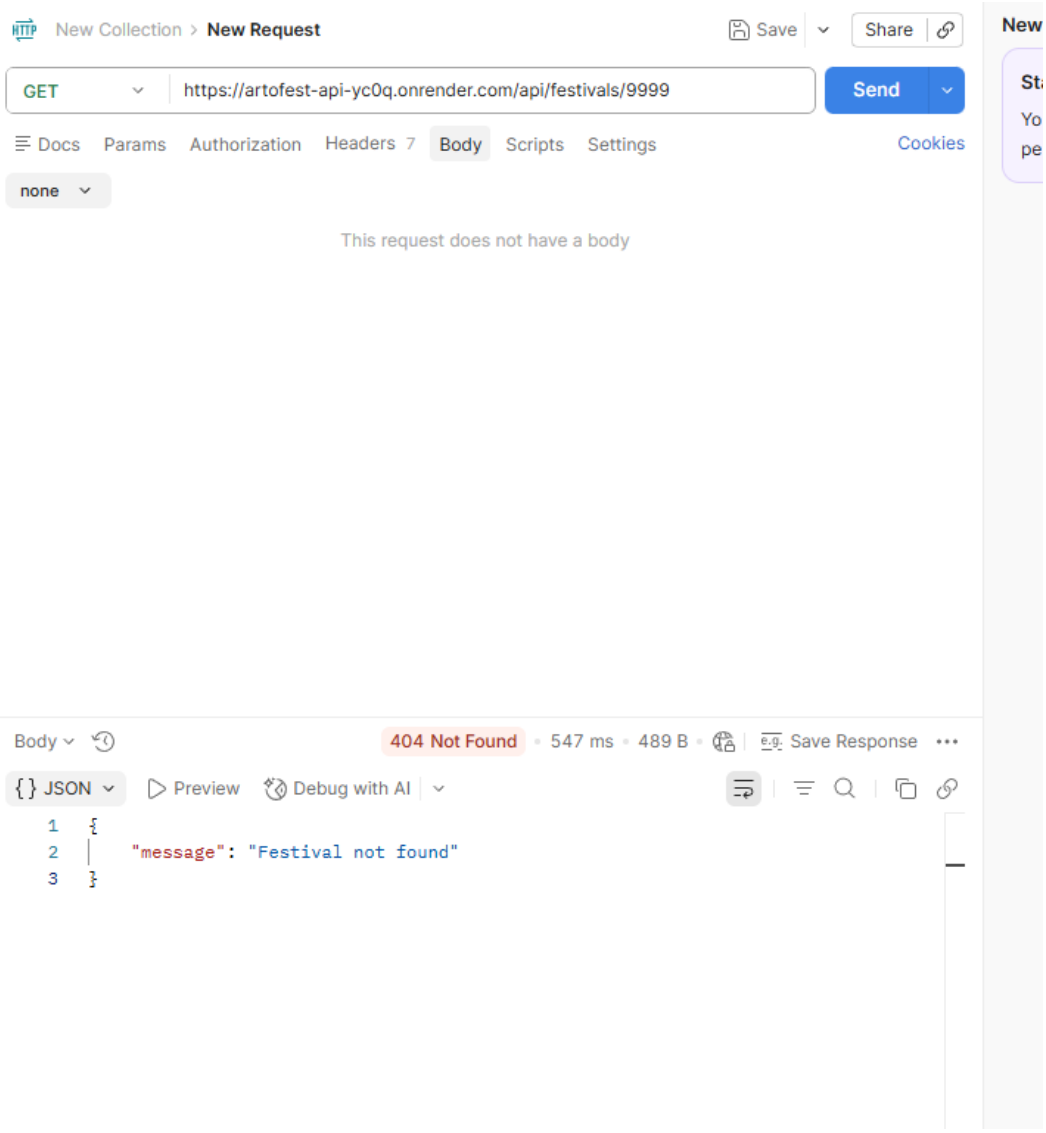
TEST 3: Testing search logic implemented and the test was performed using a particular country. Response was OK

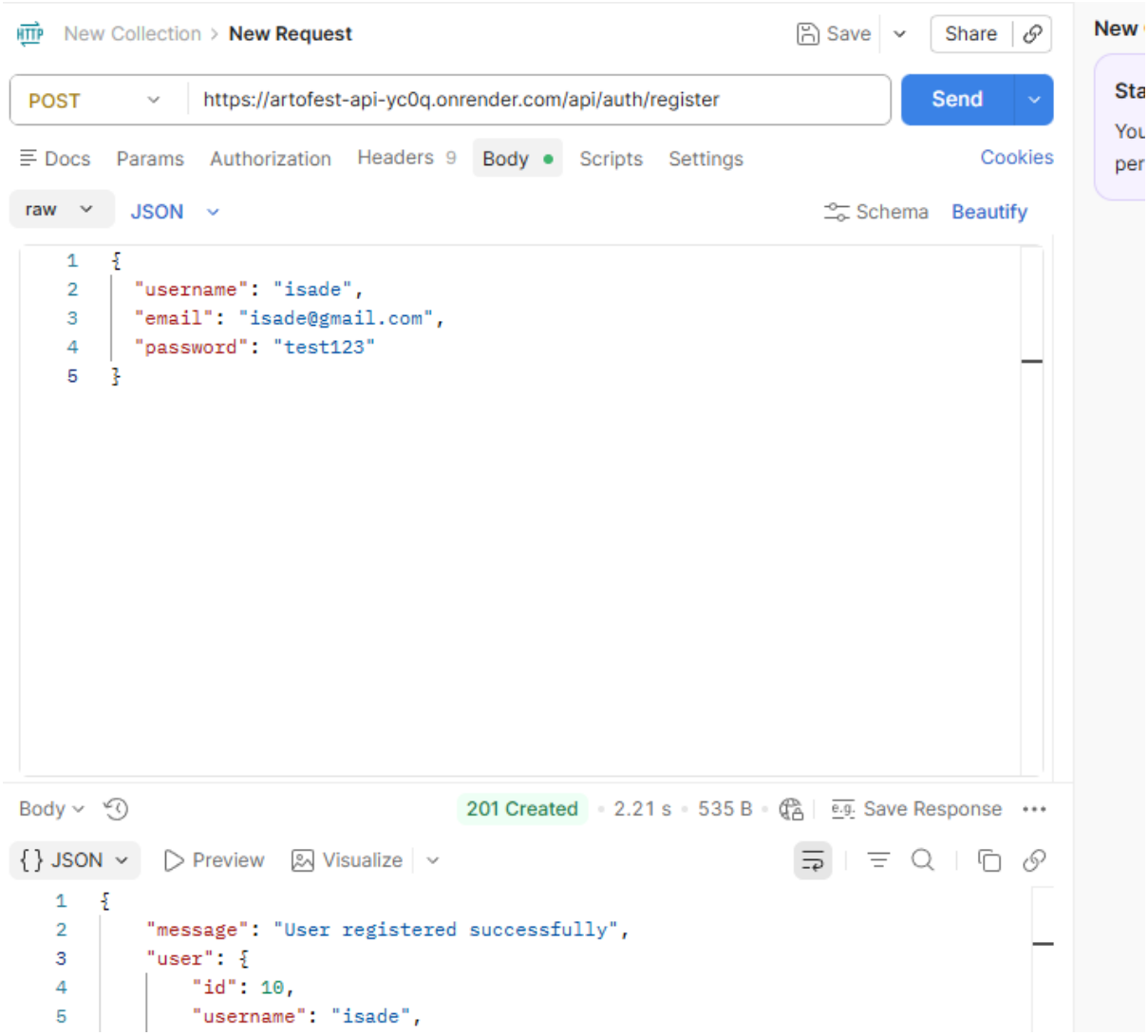


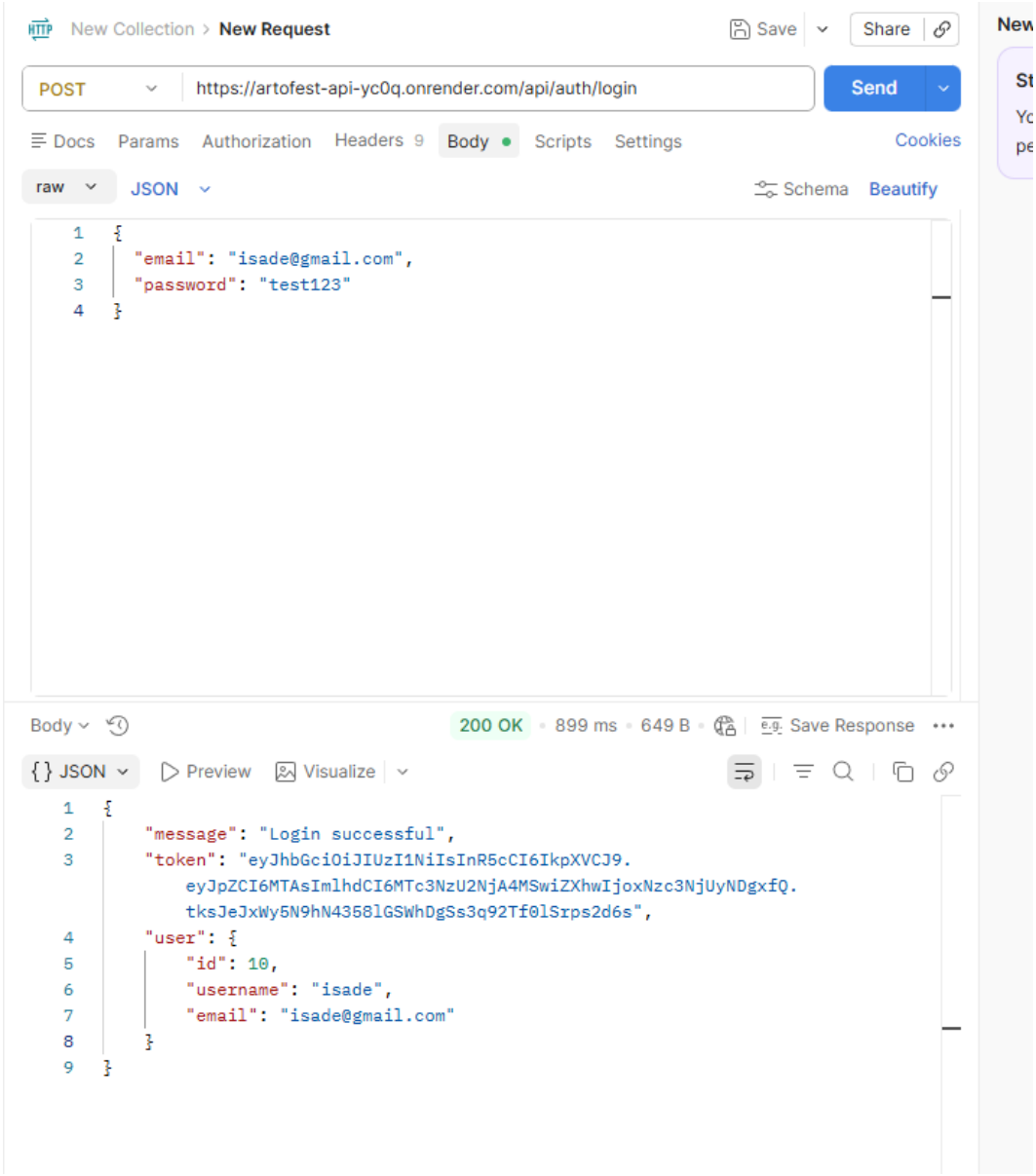
TEST 4: Testing login endpoint to refuse access to user not registered, response was positive giving the right response 401 Unauthorized.

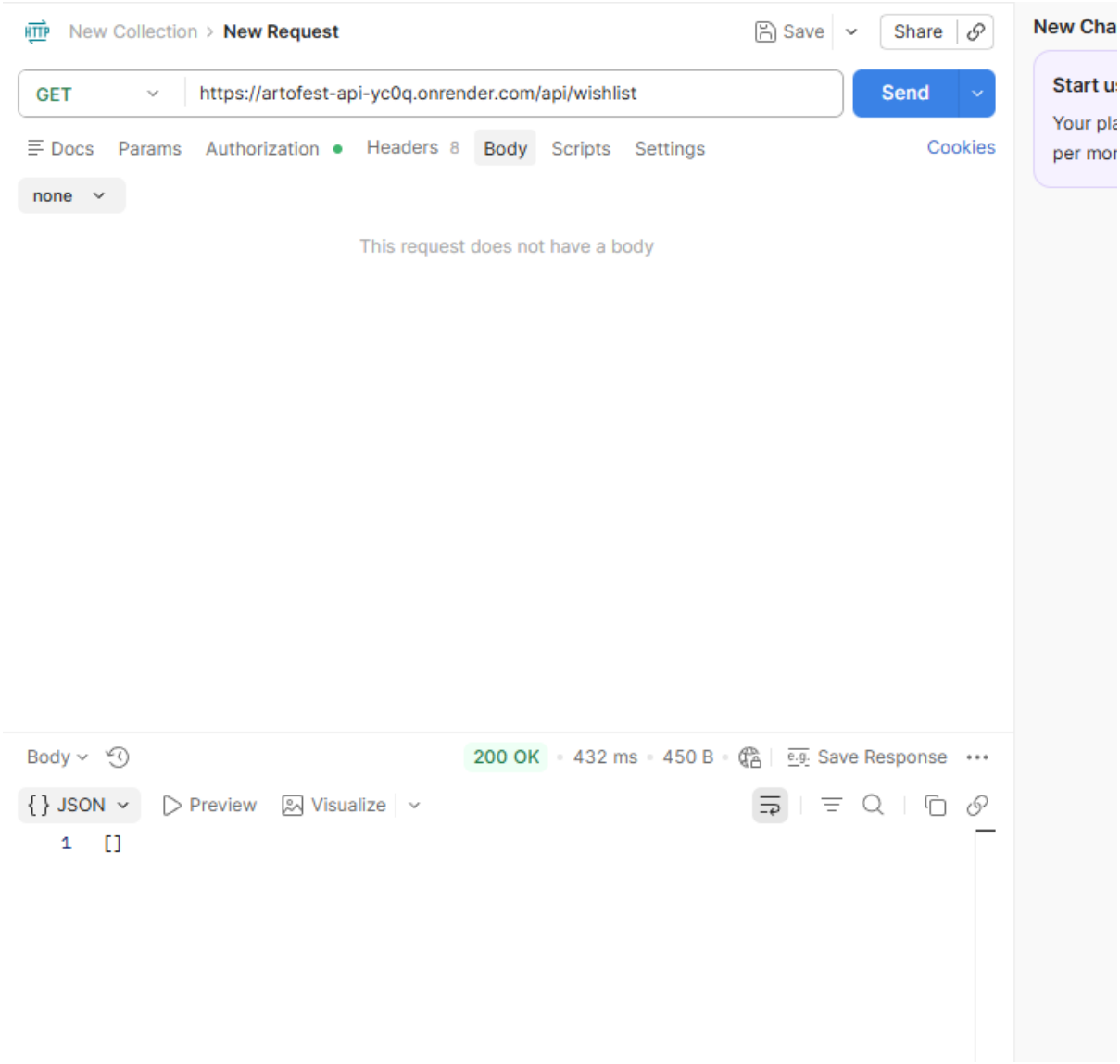

New Collection > New Request
Save
Share
POST
https://artofest-api-yc0q.onrender.com/api/auth/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
1 {
2 "email": "test@test.com",
3 "password": "123456"
4 }
Body
401 Unauthorized
609 ms
493 B
Save Response
{ } JSON
Preview
Pass the correct auth credentials
1 {
2 "message": "Invalid credentials"
3 }
New Chat
Start using
Your plan in
per month t

TEST 5: Error case for getallfestival was checked, response was positive with 404 not found.


New Collection > New Request
Save
Share
GET
https://artofest-api-yc0q.onrender.com/api/festivals/9999
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
none
This request does not have a body
Body
404 Not Found
547 ms
489 B
Save Response
{} JSON
Preview
Debug with AI
1 {
2 "message": "Festival not found"
3 }


New Collection > New Request
Save
Share
POST
https://artofest-api-yc0q.onrender.com/api/auth/register
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"username": "isade",
"email": "isade@gmail.com",
"password": "test123"
}
Body
201 Created
2.21 s
535 B
Save Response
{} JSON
Preview
Visualize
{
"message": "User registered successfully",
"user": {
"id": 10,
"username": "isade",


New Collection > New Request
Save
Share
POST
https://artofest-api-yc0q.onrender.com/api/auth/login
Send
Docs
Params
Authorization
Headers 9
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"email": "isade@gmail.com",
"password": "test123"
}
Body
200 OK
899 ms
649 B
Save Response
{} JSON
Preview
Visualize
{
"message": "Login successful",
"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJpZCI6MTAsImlhdCI6MTc3NzU2NjA4MSwiZXhwIjoxNzc3NjUyNDgxfQ.
tksJeJxWy5N9hN4358lGSWhDgSs3q92Tf0lSrps2d6s",
"user": {
"id": 10,
"username": "isade",
"email": "isade@gmail.com"
}
}



The screenshots above followed these steps below;
TEST 6: Testing the endpoint to register a user and the response was positive
TEST 7: Testing the endpoint to login registered user and the response was positive
TEST 8: Testing the endpoint of get wishlist for logged in user and the response was positive

Unit Testing of Backend Logic

Although the project primarily used manual testing, individual backend components were validated in isolation to ensure correctness.

Controller Testing

Each controller function was tested to verify:

- Correct query execution
- Proper response formatting
- Handling of edge cases

For example, the `getFestivalById` controller was tested with:

- Valid IDs
- Invalid IDs
- Missing parameters

```
if (result.rows.length === 0) {
  return res.status(404).json({ message: "Festival not found" });
}
```

This validation ensures that the API correctly handles cases where a requested resource does not exist, improving reliability and user experience.

Database Query Validation

Special attention was given to validating SQL queries, especially those involving joins and aggregations.

Key Query Features Tested

- JOIN operations across multiple tables
- Filtering logic (`ILIKE`, parameters)
- JSON aggregation (`json_agg`)
- GROUP BY correctness

```
COALESCE(
  json_agg(fi.image_url) FILTER (WHERE fi.image_url IS NOT NULL),
  '[]'
) AS images
```

Code snippet from FestivalController.

The use of `json_agg` was tested to ensure:

- Multiple images are grouped correctly
- No null values are included
- Empty arrays are returned when no images exist

This improves frontend performance by reducing the need for additional API calls.

Authentication and Security Testing

Authentication was tested to ensure secure access to protected routes.

JWT Validation Tests

- Valid token → Access granted
- Invalid token → Access denied
- Missing token → Unauthorized response

```
const token = req.headers.authorization?.split(" ")[1];
```

JWT tokens are extracted from request headers and validated before granting access to protected endpoints such as the wishlist.

Error Handling and Edge Case Testing

Robust error handling was implemented and tested to ensure system stability.

Handled Scenarios

- Invalid query parameters
- Empty database results
- Duplicate wishlist entries
- Database connection failures

```
catch (error) {
  console.error(error);
  res.status(500).json({ message: "Server Error" });
}
```

Centralized error handling ensures that unexpected failures do not crash the application and instead return meaningful responses to the client.

Validation of API Responses

All API responses were validated to ensure:

- Correct JSON structure
- Consistent field naming
- Proper nesting of data (e.g., images array)

This ensures seamless integration with the frontend.

Quality Assurance Measures

Additional quality assurance steps included:

- Testing endpoints with large datasets
- Verifying response times
- Ensuring consistent API behaviour
- Cross-checking Swagger documentation with actual responses

Testing Challenges and Solutions

Challenge 1: Duplicate Data Due to Joins

- **Solution:** Used `GROUP BY` and `json_agg`

Challenge 2: Handling Missing Images

- **Solution:** Used `COALESCE` to return empty arrays

Challenge 3: Authentication Failures

- **Solution:** Implemented JWT validation checks

Summary

The backend testing strategy ensured that all API endpoints function correctly, securely, and efficiently. Through a combination of manual testing, query validation,

and error handling, the system achieved a high level of reliability and readiness for deployment.

UI Testing – Written by Favour Akuchie

UI testing was conducted to evaluate the visual design, usability, and responsiveness of the ArtoFest interface. The goal was to ensure that users can interact with the system efficiently and without confusion.

Testing focused on key interface components, including:

- Homepage search filters
- Navigation bar
- Festival cards
- Explore page layouts
- Festival detail pages

Users were asked to perform tasks such as searching for festivals, navigating between pages, and viewing festival details. Their interactions were observed to identify usability issues.

Key Findings:

- Navigation was clear and intuitive, allowing users to move between pages easily
- The search and filtering system was effective in narrowing down results
- Festival cards were visually clear and easy to understand
- The map views improved the browsing experience

Issues Identified:

- Minor inconsistencies in spacing and alignment on some screen sizes

Improvements Made:

- Adjusted spacing and layout for better consistency

Overall, UI testing confirmed that the interface is user-friendly, responsive and aligned with the project's design goals

User Acceptance Testing – Written by Paul Oko-Jaja

User Acceptance Testing (UAT) was conducted to evaluate whether the artofest.com platform meets user needs and client requirements. The primary objective of this testing phase was to ensure that users could effectively discover, explore and interact with festival information through a clear and intuitive interface.

The testing process focused on key system features, including the home page filtering functionality, the explore page (grid and map views), festival detail pages, and overall navigation between pages. These features were selected as they represent the core functionality of the system and are critical to the user experience.

Testers were first provided with a brief walkthrough of the system, outlining the main features and expected interactions. Following this, they were asked to complete a series of tasks, such as applying filters to search for festivals, navigating between pages, and viewing detailed festival information.

The results of the testing indicated that the system performed effectively, with the majority of core features functioning as intended. Users were able to locate festivals quickly, typically within a few interactions, and navigation between pages was smooth and consistent. The explore page, including both grid and map views, was particularly well received for its clarity and usability.

Some minor issues were identified during testing, primarily relating to small user interface inconsistencies and opportunities to improve clarity in certain areas. However, no critical issues were found that significantly impacted the functionality of the system.

Based on the feedback received, minor refinements were implemented to enhance usability and improve the overall user experience. These included adjustments to interface elements and improvements to visual clarity. Overall, the UAT process confirmed that the system meets its intended objectives and is suitable for use by its target audience.

Results and Improvements – Written by Jacob Askew

The user testing feedback for ArtoFest was generally positive and did not identify any major functional issues that required immediate code changes. Both testers were able to use the website successfully and gave positive comments about the overall usability, layout and design. This suggests that the main frontend goals of clear navigation, accessible content presentation and a smooth discovery experience were largely achieved.

Tester 1 gave particularly positive feedback on the usability and functionality of the website. They described the platform as easy to navigate and stated that users could move through pages without confusion. The simple design was also seen as a strength because it made the interface clean and not overwhelming. Tester 1 also commented positively on the clear structure of the content, fast loading speeds and the map view feature, which they identified as a favourite part of the system. This feedback supports the design decision to keep the interface simple, visual and easy to move through.

Tester 2 also provided positive feedback and described the website as professional, consistent and easy to use. This supports the effectiveness of the visual design and layout decisions made during frontend development. The consistency of the navigation, footer, page structure and theme helped the application feel more complete and easier to understand.

As no major usability or functionality problems were identified during this testing round, no immediate frontend changes were made after the feedback was collected. This is important to state because the purpose of testing was not only to find faults, but also to confirm whether the implemented system met user expectations. In this case, the feedback indicated that the core frontend experience was working effectively.

However, one improvement was identified. Tester 2 noted that the image resolution could be improved. This suggests that while the layout and functionality were suitable, the visual quality of some festival images could be enhanced in a future version. This improvement would likely involve using higher-resolution images, optimising image sources, and ensuring image dimensions are more consistent across festival cards and detail pages.

The main outcome from this testing stage is that the frontend was validated as usable, professional and functional, while also identifying image quality as a future refinement. The results show that the MVP is suitable for demonstration, but further polish would improve the overall visual presentation before a public release.

Future improvements based on this testing include replacing lower-quality images with higher-resolution festival images, reviewing image scaling across cards and detail pages, carrying out further mobile testing, and collecting feedback from a larger number of users. Additional testing should also be completed after deployment, as real hosting conditions may reveal performance or layout issues that are not visible during local development.

Overall, the testing results support the conclusion that ArtoFest achieved its main frontend usability goals. The system was considered easy to navigate, visually consistent and smooth to use, with no major functional problems raised by testers. The main improvement identified was visual rather than structural, meaning the frontend is

currently functional but could benefit from further image-quality refinement before being treated as a polished public product.

Post-Project Support- Written by Idowu Adeleke

Overview

Post-project support is essential to ensure the continued reliability, performance, and stability of the Artofest backend system after deployment. This phase focuses on maintaining the API, resolving issues and ensuring the system operates efficiently in a real-world environment.

The backend system, built using Node.js and PostgreSQL, requires ongoing monitoring and maintenance to support users and ensure a seamless experience.

Bug Fixing and Issue Resolution

After deployment, users may encounter unexpected issues that were not identified during testing. A structured approach is required to handle such cases.

This includes:

Logging errors using backend logging mechanisms

Identifying the root cause of issues

Applying fixes and redeploying updates

Database Maintenance

The PostgreSQL database requires regular maintenance to ensure optimal performance and data integrity.

Maintenance activities include:

Cleaning outdated or unused data

Ensuring consistency across related tables

Monitoring query performance

Performing regular backups

These practices prevent data corruption and ensure the system remains efficient as data grows.

API Maintenance and Updates

The backend API must be maintained to ensure compatibility with the frontend and to accommodate minor updates.

This includes:

Updating endpoints when necessary

Fixing inconsistencies in responses

Ensuring backward compatibility

Careful API maintenance ensures that changes do not break existing frontend functionality.

Security Maintenance

Security must be continuously maintained even after deployment.

Key activities include:

Monitoring unauthorized access attempts

Ensuring JWT authentication remains secure

Updating dependencies to fix vulnerabilities

Regular security checks help protect user data and maintain system integrity.

Summary

Through continuous monitoring, bug fixing, database maintenance and security checks, the system can provide a reliable experience for users while adapting to real-world usage conditions.

Conclusion – Written by Jacob Askew and Paul Oko-Jaja

Product and Technical Outcomes

Our team successfully produced a functional festival discovery and planning MVP that demonstrates the main aims of the original project. The final system provides users with a central platform for browsing festivals, filtering festival information, viewing individual festival profiles, saving festivals to a personal wishlist, organising saved festivals by month, and accessing supporting features such as festival streams and prototype booking/travel guidance.

From a technical perspective, one of the main achievements was the development of a cross-platform frontend using React Native, Expo and Expo Router. This allowed the project to support a web-based interface while remaining suitable for future mobile app development from the same codebase. This directly reflects the project objective of avoiding separate native applications and instead creating a shared, maintainable solution.

The finished product also demonstrates successful frontend and backend integration. Festival data, wishlist data and stream content are retrieved from backend API endpoints rather than being hard-coded into the interface. This makes the application more realistic and easier to maintain because festival records can be updated through the backend/database without requiring major changes to the frontend pages.

A significant improvement beyond the original project scope was the implementation of account-based functionality. Although the initial project plan excluded user login and account management from the first release, the final MVP includes registration, login, stored sessions, profile handling and protected wishlist functionality. This strengthened the planning side of the application because users can save festivals to their own account instead of only browsing public festival information.

The project also delivered a clearer user experience through a custom top navigation bar, consistent footer, responsive layouts, searchable festival cards, map-based exploration and individual festival detail pages. These features help the system feel like a complete product rather than a static prototype. The Plan page in particular supports the project's purpose by allowing users to build a personal festival wishlist and organise saved festivals by month

However, not every planned feature was completed to full production standard. The AI planner remains a placeholder because of time constraints and limited free AI service options. The Book page is also a proof-of-concept rather than a live booking system, as real ticket, travel or accommodation booking would require external providers, live availability, legal considerations and further development time. These limitations are clearly presented in the application so that users are not misled about unfinished functionality.

Overall, the technical outcome of ArtoFest is a working MVP that supports the most important user journeys: discovering festivals, viewing festival details, creating an account, saving festivals, organising a wishlist and viewing related stream content.

Project Delivery and Client Reflection

From a project management perspective, ArtoFest progressed from an initial concept into a working MVP through structured planning, role allocation, iterative development and regular communication. The project began with a clear focus on solving the problem of fragmented festival discovery and planning, where users often need to search across multiple platforms to find festival information, official websites, travel options and related media.

The team established a project plan that defined the intended scope, objectives, stakeholders, resources, risks and communication approach. This provided a foundation for development by identifying the main features that the system should aim

to provide, including festival browsing, search and filtering, individual festival information, wishlist planning, cross-platform support and future booking/travel integration.

During development, the project was managed using an Agile-inspired approach. Work was divided into sprint-based tasks so that planning, design, development, testing and refinement could be handled progressively rather than attempted all at once. This helped the team break down the project into manageable stages and allowed priorities to be adjusted as technical constraints became clearer.

Client communication was an important part of the project delivery process. Requirements and expectations were clarified through meetings and communication with the client representatives, Nadia Krasteva and Russell Howe. Their feedback helped shape the direction of the platform and supported the decision to focus on a user-friendly festival discovery and planning experience rather than only producing a basic information directory.

The project also required active risk management. Some features originally discussed during planning, such as full booking integration, travel/accommodation APIs and AI-assisted planning, proved difficult to complete fully within the available time and resource constraints. CMS functionality was partly achieved through a prototype Admin Section, allowing server-approved admin accounts to create, update and delete festival and stream records through backend-protected endpoints. However, further refinement would be needed before this could be considered a complete production-ready administration system. These risks were managed by focusing on a realistic MVP and clearly separating completed functionality from prototype or future-phase features. This allowed the team to still deliver a working product while avoiding misleading claims about unfinished functionality.

The final MVP reflects the core direction of the original project while also showing how the scope evolved during development. The system successfully demonstrates festival discovery, backend-connected festival data, user authentication, wishlist planning, stream content and responsive navigation. While some advanced features remain future work, the project still provides a strong basis for continued development and client review.

Overall, the project delivery process showed the importance of planning, communication, prioritisation and scope control. ArtoFest developed into a functional prototype that can be demonstrated to the client and used as a foundation for further improvements. Future work should focus on deployment, further user acceptance testing, client feedback, refining the existing administrative tools and expanding prototype features into production-ready services.

References

European Festivals Association (2026) *FestivalFinder.eu*. Available at: https://www.festivalfinder.eu/ (Accessed: 2026).

Express.js (2026) *Express - Node.js web application framework*. Available at: https://expressjs.com/

Expo (2026) *Expo documentation*. Available at: https://docs.expo.dev/

Expo (2026) *Introduction to Expo Router*. Available at: https://docs.expo.dev/router/introduction/

JSON Web Token (JWT) (2026) *Introduction to JSON Web Tokens*. Available at: https://jwt.io/introduction

Leaflet (2026) *Leaflet API reference*. Available at: https://leafletjs.com/reference.html

Meta Platforms, Inc. (2026) *Introduction: React Native*. Available at: https://reactnative.dev/docs/getting-started

Node.js (2026) *Introduction to Node.js*. Available at: https://nodejs.org/learn/getting-started/introduction-to-nodejs

OpenAPI Initiative (2025) *OpenAPI Specification v3.2.0*. Available at: https://spec.openapis.org/oas/v3.2.0.html

OWASP Foundation (2026) *Authentication Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

OWASP Foundation (2026) *Session Management Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Foundation (2026) *SQL Injection Prevention Cheat Sheet*. Available at: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

PostgreSQL Global Development Group (2026) *PostgreSQL documentation: Aggregate functions*. Available at: https://www.postgresql.org/docs/current/functions-aggregate.html

PostgreSQL Global Development Group (2026) *PostgreSQL documentation: JSON functions and operators*. Available at: https://www.postgresql.org/docs/current/functions-json.html

Swagger (2026) *What is OpenAPI?* Available at: https://swagger.io/docs/specification/v3_0/about/

Appendix (Personal Reflections Below)

Appendix A – Personal Evaluation / Feedback / Reflection

Idowu Adeleke (Backend Developer)

Throughout the Artofest project, I developed a strong foundation in backend development, significantly enhancing both my theoretical understanding and practical implementation skills. I gained hands-on experience in designing and building RESTful APIs using Node.js and Express, alongside developing and managing a relational database using PostgreSQL. A key technical advancement was my ability to construct complex SQL queries involving multiple joins across interconnected tables such as festivals, genres, and festival_images. This required not only technical precision but also a deeper understanding of relational data modelling. Furthermore, I implemented PostgreSQL's `json_agg` function to aggregate related image data into structured JSON arrays, which improved both the efficiency and clarity of API responses. This demonstrated my ability to move beyond basic database operations and apply more advanced query optimisation techniques.

In addition, I strengthened my understanding of authentication mechanisms by implementing JSON Web Tokens (JWT) to secure protected endpoints. This allowed me to critically engage with the concept of stateless authentication and its advantages in scalable web applications. I also developed proficiency in using industry-standard tools such as Postman for API testing and Swagger for API documentation. These tools not only improved the quality and reliability of my implementation but also highlighted the importance of clear communication between backend and frontend systems in a collaborative development environment.

Beyond technical competencies, this project contributed significantly to my development of essential soft skills. Communication became a critical component of my role, as I needed to ensure that API responses aligned precisely with frontend requirements. This required continuous dialogue with the frontend engineer (Jacob) and an ability to translate technical backend structures into usable data formats. Collaboration with the UI/UX designer (Favour) and project manager (Paul) further enhanced my teamwork skills, as I had to adapt to design requirements and project timelines while maintaining backend consistency. I also developed stronger time management skills, particularly in balancing coding, testing, and documentation tasks within project deadlines. Importantly, I cultivated a more analytical problem-solving approach, enabling me to break down complex challenges into structured and manageable solutions.

One of the most significant challenges I encountered was managing relational data efficiently. Initially, retrieving festival data alongside multiple associated images resulted in redundant queries and inefficient data handling. This limitation prompted a deeper evaluation of database optimisation techniques, leading to the implementation of `json_agg` to consolidate related data into a single structured response. This not only improved performance but also reduced the complexity of frontend data handling. Another challenge involved implementing dynamic filtering functionality, where SQL queries needed to adapt based on optional user inputs such as country, genre, and search keywords. Ensuring both flexibility and security required the use of parameterised queries, reinforcing my understanding of best practices in preventing SQL injection vulnerabilities. Additionally, debugging issues related to incorrect joins and empty datasets required systematic testing and critical analysis of query logic, further strengthening my troubleshooting abilities.

My experience working within the team was largely positive and highlighted the importance of role clarity in collaborative projects. Each team member contributed within their area of expertise, which facilitated efficient task distribution. My interaction with Jacob was particularly important, as it ensured that backend data structures were compatible with user interface requirements. This collaboration emphasised the interdependence between system components and the need for consistency across the application. Paul provided structure and ensured adherence to deadlines, while the Favour influenced how data needed to be presented, indirectly shaping backend responses. While communication was generally effective, this experience also highlighted the importance of early alignment between frontend expectations and backend implementation to minimise later adjustments.

Although this project was conducted in an academic setting, it effectively simulated a real-world client scenario. One of the key lessons I learned was the importance of thoroughly understanding requirements before beginning implementation. Misinterpretation at an early stage can lead to inefficiencies and rework later in the development process. I also recognised that backend development extends beyond technical implementation; it involves delivering solutions that are efficient, scalable, and aligned with user needs. This required me to think critically about data structure, response performance, and usability from a broader system perspective. Additionally, I learned the importance of designing systems with flexibility in mind, as requirements may evolve over time and necessitate future modifications.

A particularly valuable learning experience occurred when I encountered issues with duplicate data being returned due to improper SQL joins. This issue exposed a gap in my understanding of how relational queries can unintentionally produce redundant results. Addressing this required the introduction of GROUP BY clauses alongside JSON aggregation, which resolved the issue and improved the overall data structure. Another

significant improvement involved implementing structured error handling. Initially, the API lacked clear feedback mechanisms, which could lead to confusion during integration and testing. By introducing appropriate HTTP status codes and consistent error responses, I enhanced both the reliability and usability of the system. These experiences reinforced the importance of writing robust, maintainable code and adopting a proactive approach to testing and debugging.

Looking forward, I intend to build upon the skills developed during this project by exploring more advanced backend concepts. Areas of focus include automated testing frameworks, API performance optimisation through caching strategies, and scalable deployment using cloud-based infrastructure. I am also interested in gaining a deeper understanding of microservices architecture and how it can be applied to large-scale systems. In addition, I plan to continue developing personal projects to further strengthen my practical experience and build a professional portfolio that reflects my backend development capabilities.

In conclusion, the Artofest project provided a comprehensive and challenging learning experience that significantly enhanced both my technical and professional skills. It enabled me to move beyond theoretical knowledge and apply backend development concepts in a practical context, while also developing critical thinking, collaboration, and problem-solving abilities. This experience has not only strengthened my confidence in backend development but has also prepared me for more complex, real-world software engineering challenges.

Appendix B– Personal Evaluation / Feedback / Reflection

Favour Akuchie (Web Designer)

The evaluation of the ArtoFest project focuses on the effectiveness of the user interface design, the overall user experience,and the extent to which the design met both user needs and project objectives. As the Web Designer, my primary responsibility was to ensure that the platform was visually coherent, intuitive to use, and supportive of the core user journey of discovering and planning festival experiences.

One of the most successful aspects of the design is the overall consistency of the interface. A unified layout was maintained across all major pages, including the homepage, explore page, festival detail pages, and planning section. The implementation of a global navigation bar and footer contributed significantly to this consistency, allowing users to move between different sections of the application with ease. This design decision aligns with established usability principles, particularly

consistency and predictability, which are essential for reducing user confusion and improving efficiency.

The homepage design proved to be an effective entry point into the system. By combining a visually engaging hero section with clearly defined filter controls, the design encourages immediate interaction. The use of dropdown filters for country, month, and art form allows users to refine their search in a structured way, which helps prevent information overload. This approach demonstrates an understanding of progressive disclosure, where users are presented with manageable amounts of information rather than being overwhelmed by large datasets.

The Explore page further enhances the user experience by offering both grid and map-based views. This dual presentation was a deliberate design choice aimed at supporting different user preferences. While the grid view allows for quick visual scanning of multiple festivals, the map view provides geographical context, which is particularly useful for users planning travel. This flexibility improves usability and reflects a user-centred design approach. Additionally, grouping festivals by country and providing filtering options helps users navigate the platform more efficiently.

The design of festival cards and detail pages also contributes positively to the system. Festival cards present key information in a concise and visually appealing format, allowing users to quickly identify relevant events. The transition from these summary cards to detailed festival pages follows a clear information hierarchy, where users are progressively provided with more in-depth information. This supports informed decision-making and aligns with good UX design practices.

Another key strength is the responsiveness of the interface. The system was designed to function across both desktop and mobile platforms, and the layout adapts effectively to different screen sizes. Navigation elements, content structure, and layout spacing adjust to maintain usability on smaller screens. This cross-platform responsiveness was an important requirement of the project and ensures that the application remains accessible to a wider audience.

However, despite these strengths, several limitations were identified during evaluation. One of the main areas for improvement is user feedback during interactions. While basic feedback mechanisms such as loading indicators and error messages are present, some interactions lack clear visual confirmation. For example, when users apply filters or add festivals to their wishlist, the feedback could be more immediate and visually distinct. Enhancing these interaction cues would improve user confidence and make the interface feel more responsive.

The booking and travel section highlights a limitation in terms of functionality rather than design. While the interface for this feature is visually consistent with the rest of the system, it operates as a prototype and does not provide real booking capabilities. From a user experience perspective, this may create a gap between user expectations and actual functionality. However, this limitation was acknowledged during development, and the inclusion of a “booking unavailable” page helps manage user expectations transparently.

Similarly, the AI planning feature remains incomplete and is currently represented as a placeholder within the interface. While its inclusion demonstrates forward planning and innovation, the lack of implementation limits its contribution to the overall user experience. Completing this feature in future iterations would significantly enhance the platform by providing personalised recommendations and planning support.

From a usability standpoint, the system performs well in supporting its core functions. Users are able to search for festivals, navigate between pages, and access detailed information with relative ease. The filtering system is particularly effective in helping users narrow down options. However, further usability testing with a larger and more diverse group of users would provide deeper insights into potential improvements, particularly in terms of accessibility and interaction design.

Accessibility is another area where improvements could be made. While the current design meets basic usability standards, additional considerations such as improved colour contrast, keyboard navigation support, and compatibility with assistive technologies would make the platform more inclusive. Incorporating accessibility best practices would not only improve usability for a wider audience but also align the system with modern web standards.

Reflecting on my role as the Web Designer, this project provided valuable experience in applying UI/UX principles within a real development environment. One of the key lessons learned was the importance of balancing visual design with functionality. While creating an aesthetically pleasing interface is important, it must always support usability and user goals. I also gained a deeper understanding of responsive design and the challenges involved in creating a consistent experience across multiple platforms.

In conclusion, the ArtoFest interface successfully meets its primary objectives by providing a clear, intuitive, and visually engaging platform for festival discovery and planning. The design demonstrates strong usability, consistent structure, and effective information hierarchy. However, there are areas for improvement, particularly in interaction feedback, accessibility, and the completion of planned features. Overall, the project provides a solid foundation for future development and highlights the importance of iterative design and continuous user-focused improvement.

Appendix C – Personal Evaluation / Feedback / Reflection

Paul Oko-Jaja (Project Manager)

Looking back at this module, I can honestly say it has been one of the most challenging but also most valuable experiences in my course so far. At the start, I didn't fully understand how all the different parts of a software project come together, but working through this project step by step has given me a much clearer picture of the full development process.

In terms of technical skills, I feel like I've improved a lot, especially in how I approach building features rather than just writing code. For example, when working on parts like the explore page, I had to think beyond just making it work and instead consider how users would interact with it. This included decisions around layout, filtering, and how information is displayed. It made me realise that good development is not just about functionality, but also about usability and design.

I also became more comfortable using tools like Git and Trello. At the beginning, I didn't use them as effectively, but over time I understood how important they are for keeping everything organised, especially when multiple people are working on the same project. Using sprints also helped me break work down into smaller, manageable tasks, which made the overall project feel less overwhelming.

One area that was completely new to me was testing, particularly User Acceptance Testing (UAT). At first, I didn't really understand what was expected, and it felt quite confusing trying to structure test cases properly. However, once I got into it, I started to see its importance. It changed how I think about development, as I began to consider how features would be tested while I was building them, not just afterwards.

In terms of soft skills, I think communication was the biggest area of growth for me. Working in a group is very different from working individually, and there were times when things weren't clearly communicated or when people had different ideas about how something should be done. Instead of avoiding those situations, I learned to speak up more and listen properly to others. Over time, this made our teamwork smoother.

Time management was another challenge for me. There were moments where I underestimated how long tasks would take, especially when dealing with bugs or unexpected issues. This sometimes put pressure on me closer to deadlines. However, it taught me to plan more realistically and start tasks earlier rather than leaving them too late.

One of the biggest challenges overall was dealing with uncertainty. Not everything was clearly defined, and sometimes we had to figure things out as we went along. This was frustrating at times, but it also helped me become more independent and better at problem-solving. Instead of relying on step-by-step instructions, I had to think things through myself or work with my team to find solutions.

My experience with my group mates was generally positive. We all contributed in different ways, and even though there were a few moments where communication could have been better, we managed to support each other and keep the project moving forward. I realised that a good team is not one where everything is perfect, but one where people are willing to cooperate and adapt when needed.

Even though we didn't work with a real client in a full professional sense, thinking from a client's perspective was still an important part of the project. It made me realise that the end user should always be the focus. It's easy to get caught up in technical details, but if the system is not easy to use, then it doesn't fully meet its purpose.

One experience that stood out to me was working on the documentation, especially the UAT and final report. At first, I found it difficult to understand what was expected and how detailed everything needed to be. It felt quite overwhelming, but as I worked through it, I became more confident in writing in a structured and professional way. This is something I know will be useful in the future.

Another important moment was towards the end of the project when deadlines were approaching. There was pressure to make sure everything was complete and working, which meant prioritising the most important features. This experience taught me that in real projects, it's not always about making everything perfect, but about delivering something functional and reliable on time.

Going forward, I want to continue improving both my technical and soft skills. Technically, I want to become more confident in building full systems and handling more complex features. I also want to improve my debugging and testing skills, as I now understand how important they are. In terms of soft skills, I want to keep improving my communication and time management, as these had a big impact on my experience in this project.

Overall, this module has helped me grow a lot, not just as a developer but as someone working in a team. It showed me what working on a real project is like, including the challenges and the learning process that comes with it. Even though there were difficult moments, I feel more prepared for future projects because of this experience.

Appendix D – Personal Evaluation / Feedback / Reflection

Jacob Askew (Front End Engineer)

This project has been one of the most valuable technical experiences I have had during the course so far because it required me to move beyond isolated programming tasks and build a larger, connected system that had to work as part of a real group project. My main responsibility was frontend development, and I took primary responsibility for implementing the user-facing coding element of ArtoFest. This included building the React Native/Expo application, creating the page structure, connecting the frontend to the backend API, implementing navigation, developing festival search and detail pages, adding authentication flows and building the wishlist/planning functionality.

One of the main technical skills I developed was working with React Native and Expo in a more complete project environment. Before this project, I had experience with programming and web development, but this was a much larger application with multiple pages, shared components, reusable utility files and platform-specific behaviour. I became more confident using Expo Router for file-based navigation, creating dynamic routes such as festival detail pages, and managing layout across both desktop and mobile screen sizes. I also gained more experience with TypeScript, especially around defining data types, handling optional API values, and preventing errors caused by missing or inconsistent data.

A major technical achievement for me was API integration. The frontend was not just a static design; it retrieved festival data, stream content, login responses, and wishlist information from backend endpoints. This helped me understand how important it is for frontend and backend development to be aligned. I had to work with data returned by the backend, clean it, validate it and display it safely in the interface. I also implemented reusable helper functions for tasks such as normalising strings, formatting dates, validating festival IDs, building website links and handling month filtering. This made the frontend more maintainable because the same logic could be reused across different screens instead of being duplicated.

Another important skill I developed was authentication handling. The final application includes registration, login, stored sessions, profile handling, and a protected Plan page. Implementing this helped me understand how a frontend application uses token-based authentication in practice. The Plan page checks whether the user is logged in before loading personal wishlist data, and authenticated requests use the saved token when communicating with the backend. This was a useful experience because it

showed me the difference between public features, such as festival browsing, and account-specific features, such as saving festivals to a personal wishlist.

I also improved my ability to build responsive and user-focused interfaces. ArtoFest needed to work as both a web platform and a future mobile application, so I had to think carefully about layout, navigation, spacing, scroll behaviour, footer positioning, and how pages would appear on different screen sizes. The custom navigation bar and global footer were examples of this, as they needed to feel professional on desktop while still remaining usable on mobile. I also learned that small visual issues, such as dropdown positioning or footer behaviour, can have a big impact on the user experience.

In terms of soft skills, this project improved my problem-solving, independence, communication, and time management. I often had to work through technical issues by breaking them down step by step, checking the actual code, testing changes and avoiding assumptions. I also had to make practical decisions when certain ideas were not realistic within the time available. For example, the AI planner was originally intended to be part of the system, but due to time constraints and limited free AI options, it remained a placeholder. Similarly, the booking/travel page became a proof-of-concept rather than a live booking system because real booking functionality would require third-party providers, live availability, and additional legal and technical work.

One of the main challenges I overcame was managing the size and complexity of the frontend. As more features were added, it became more important to keep the code organised and avoid breaking existing pages. The application included homepage search, Explore, maps, festival details, login, signup, profile, wishlist planning, stream content, footer pages, and prototype booking functionality. Keeping these areas consistent while still making progress was difficult, but it helped me learn how to think more like a developer working on a full product rather than a single assignment file.

My experience with the group was mixed but useful. Each member had a defined role, with project management, backend, design, and frontend responsibilities separated across the team. From my side, I took responsibility for delivering the frontend implementation and sometimes had to work independently to keep development moving. This gave me a stronger sense of ownership over the product, but it also showed me how important balanced contribution, communication and clear handover are in group software projects. In future group work, I would want clearer technical checkpoints earlier in the project so that integration issues and unfinished features are identified sooner.

Working with clients also taught me an important lesson about scope. At the start of the project, it is easy to list many ambitious features, but during development it becomes

necessary to decide what can realistically be completed to a working standard. This project helped me understand that it is better to deliver a clear and honest MVP than to claim unfinished features are complete. The final product still demonstrates the main user journey of discovering festivals, viewing festival details, saving festivals, and planning around a wishlist, while more advanced features can be treated as future improvements.

A valuable incident during the project was implementing the wishlist feature. This required the frontend, backend, authentication system, and user interface to work together. The user needed to be logged in, the token needed to be available, the backend request needed to be authorised, and the saved festival cards needed to update correctly. This was challenging but valuable because it was one of the clearest examples of full-stack integration in the project.

After this module, I want to continue improving as a developer by becoming stronger at planning, testing, deployment, and writing cleaner code earlier in the process. I also want to improve my understanding of backend integration, security, and production deployment so that future projects can move from local development to hosted public systems more smoothly. Overall, this project has increased my confidence because I was able to build a substantial frontend application with real API integration and account-based functionality. It also showed me the importance of honest evaluation, realistic scope control and clear evidence when presenting a software project professionally.

Appendix E Testing Evaluation Document

Tester 1 Evaluation

Tester 1 provided very positive feedback regarding the usability and functionality of the website. They found the platform **easy to navigate**, highlighting that users can move through pages without confusion. The **simple design** was also appreciated, as it made the interface clean and not overwhelming.

The tester described the website as **interesting**, with **clear and well-structured content**. They specifically noted that the descriptions were concise and “straight to the point,” which prevented users from having to read excessive information at once. This indicates that the content presentation and hierarchy were effective.

In addition, performance was seen as a strong point, with the tester mentioning **fast loading speeds**, which improves overall user experience. The **map view feature** was also highlighted as a favourite element, suggesting that interactive components added significant value.

Overall, Tester 1’s feedback shows that the website is user-friendly, efficient, and well-designed, with strong navigation, clarity, and functionality.

Tester 2 Evaluation

Tester 2 also gave positive feedback, indicating overall satisfaction with the website. They described the design as **nice**, **professional**, and **consistent**, suggesting that the visual appearance meets standard expectations and maintains uniformity across pages.

The tester found the website **easy to use**, reinforcing the idea that the interface is intuitive and accessible for users. This supports the effectiveness of the design and layout decisions.

However, Tester 2 identified a minor area for improvement, noting that **image resolution could be better**. Improving the quality of images would enhance the visual appeal and overall presentation of the website.

Despite this, the tester confirmed that the system meets expected standards and provides a smooth user experience.